INDUSTRIAL STRENGTH SOFTWARE SERVICES

TECHNICAL REPORT · JK-TR-001 · REV 0.1

JOTAKEY

A FORMALLY SPECIFIED FIDO2 SECURITY KEY

AUTHOR

Javier

CONTACT

jfernand@me.com

ISSUED

2026-10-14

(C) JAVIER FERNANDEZ @ JOTASOFT 2026

ABSTRACT

Jotakey is a hardware security key, in the family of the YubiKey, written entirely in Rust. It runs on its own board, the jotakey key (an STM32L432 microcontroller with an OPTIGA Trust M secure element), and on an Espressif ESP32-C6 DevKit. This report has three aims. First, it explains from first principles how FIDO2, WebAuthn and **passkeys** work, and why they defeat phishing where passwords and one-time codes fail. Second, it documents the design of Jotakey: its threat model, key hierarchy, transports, and the honest limits of each platform against an attacker who holds the device. Third, it describes how the device is **assured**. Unit and property tests, coverage-guided fuzzing, mutation testing, and a test firmware that runs on the chip itself are all covered. So is formal verification in the Lean 4 theorem prover. The report assumes no prior knowledge of formal methods, and its longest chapter is a self-contained introduction to them, built on the proofs that actually ship with the device.

DOCUMENT CONTROL

STATUS	MVP firmware on the ESP32-C6: CTAP 2.0, non-resident credentials, USB (via bridge) and BLE. Jotakey key: hardware 0.1.0; firmware port next
HARDWARE	Jotakey key, hardware 0.1.0 (STM32L432KC + OPTIGA Trust M V1, USB-A); Espressif ESP32-C6-DevKitC-1 / M-1
REPOSITORY	`crates/` (Rust), `formal/` (Lean 4), `docs/` (this report)
NORMATIVE RULES	`AGENTS.md` — security invariants 1–9

1 Goals and threat model 4
1.1 What we are building 4
1.2 Why this needs to be right 4
1.3 Threat model 4
1.4 Security invariants 5
2 Cryptography in one chapter 6
2.1 Hash functions 6
2.2 Message authentication codes 6
2.3 Digital signatures: ECDSA on P-256 6
2.4 Randomness 7
2.5 Constant time 7
3 From passwords to passkeys 7
3.1 What is wrong with passwords 7
3.2 Public-key login 7
3.3 A short history 8
3.4 Passkeys, precisely 8
3.5 The three parties 9
4 WebAuthn: the two ceremonies 9
4.1 Registration 9
4.2 The RP ID and why phishing fails 10
4.3 authenticatorData 10
4.4 Authentication 10
4.5 Attestation 11
5 CTAP2: talking to the authenticator 11
5.1 CBOR, and why strictness matters 11
5.2 Commands 11
5.3 CTAPHID: CTAP over USB 12
5.4 FIDO over BLE 12
6 Jotakey's architecture 13
6.1 The ESP32-C6 board, and the USB problem 13
6.2 Software layers 14
6.3 The key hierarchy 14
6.4 Credential IDs: keys by derivation 15
6.5 Persistent state 15
6.6 User experience 16
6.7 The USB bridge in detail 16
7 Hardware security 20
7.1 What the chip provides 20
7.2 Provisioning 21
7.3 Side channels 21
7.4 Fault injection: the limit of this chip 21
7.5 The jotakey key: a secure element 22
7.6 The jotakey key board 23
8 Testing: five nets, different holes 24
8.1 Unit, scenario and property tests 25
8.2 Fuzzing 25
8.3 Mutation testing 26
8.4 On-device test firmware 26
8.5 End to end 27
8.6 Static checks 27
9 Formal verification, from the ground up 27
9.1 What a proof buys you 27
9.2 The landscape of techniques 28
9.3 Lean 4 in twenty minutes 28
9.4 Modelling: deciding what to prove about 30
9.5 Case study: CBOR has exactly one encoding 30

9.6 Case study: the security state machines 30
9.7 Closing the gap between model and code 32
9.8 Differential testing, step by step 32
9.9 What remains trusted 35
9.10 Working with the proofs 35
10 The proofs, one by one 35
10.1 Before you start 37
10.2 Bytes and numbers 38
10.3 CBOR heads 39
10.4 The strict validator 41
10.5 CTAPHID reassembly 42
10.6 The authenticator state machine 43
10.7 Credential IDs 45
10.8 What the proofs leave out 45
10.9 Rust and Lean names 46
11 Roadmap 48
11.1 Full CTAP 2.1 passkeys 48
11.2 Hardware 48
11.3 Assurance 49
A Invariant traceability 49
B Repository map 50
C Getting started 50
D Glossary 50
E References 51

01 GOALS AND THREAT MODEL

1.1 WHAT WE ARE BUILDING

A **security key** is a small device that holds private keys and uses them to prove the holder's identity to websites, without ever letting those keys out. You plug it in (or bring it close, or pair it over Bluetooth), the browser asks it to sign a challenge, a light blinks, you touch it, and you are logged in. The YubiKey is the best-known example.

Jotakey does the same job on two platforms. The **jotakey key** is its own hardware: a USB-A dongle with an STM32L432 microcontroller, which talks to the computer directly as a USB security key, and an Infineon **OPTIGA Trust M** secure element, which holds the root secret. A capacitive touch pad is its "touch" sensor and a red LED its only display (`hardware/HARDWARE.md`). The **ESP32-C6 DevKit** is an inexpensive, open development board: a 160 MHz RISC-V microcontroller with Wi-Fi 6, Bluetooth LE and 802.15.4 radios, 512 KiB of SRAM and 4–8 MiB of flash. It has two USB-C ports and two buttons, **BOOT** and **RESET**, plus one addressable RGB LED; Jotakey uses the BOOT button as its touch sensor. The DevKit has no secure element and needs a host bridge for USB (§6.1), so it serves as the development platform, and the jotakey key is the device to carry.

PROTOCOL	FIDO2: CTAP 2.0 authenticator, WebAuthn-compatible
ALGORITHMS	ES256 (ECDSA P-256 with SHA-256), HMAC-SHA256, HKDF-SHA256
TRANSPORTS	CTAPHID over USB (native on the key; through a host bridge on the C6), FIDO over BLE (C6)
CREDENTIALS	Non-resident ("server-side") credentials, unlimited in number
USER PRESENCE	Touch pad (key) or BOOT button (C6), debounced, fresh press required for every signature
ROOT OF TRUST	OPTIGA Trust M secure element (key); eFuse HMAC key (C6)
LANGUAGE	Rust, `no_std`, no heap in the protocol core, no `unsafe` in the core
VERIFICATION	Lean 4 models and proofs of the security-critical state machines

1.2 WHY THIS NEEDS TO BE RIGHT

A security key that is subtly wrong is worse than none. Its user believes they are protected. Three classes of failure matter:

1. **Key compromise.** If an attacker learns a credential's private key, they can log in as the user from anywhere, forever, and the user cannot tell.
2. **Silent use.** If the key can be made to sign without the user's knowledge (no touch, or a touch meant for something else), an attacker who controls the computer can log in to any account registered with the key.
3. **Denial and confusion.** Crashes, lockups or ambiguous parsing let a malicious host wedge the device, or worse, make two components disagree about what was signed.

The rest of this report is about making these failures impossible by construction where we can, unlikely where we cannot, and **documented** everywhere.

1.3 THREAT MODEL

We assume a strong attacker and list exactly what the device does and does not resist.

ATTACKER	CAPABILITIES	OUTCOME FOR JOTAKEY
Phishing site	Lures the user to `examp1e.com` and relays their login to `example.com`.	**Defeated.** Credentials are bound to the RP ID. The browser enforces the origin; the key enforces the binding (§4.3, invariant 3).
Malware on the host	Full control of the computer and browser: can send any CTAP message at any time, read every response.	**Contained.** It cannot extract keys. It can ask for signatures, but each one needs a fresh physical touch (invariant 1). It can trick the user into touch-

ATTACKER	CAPABILITIES	OUTCOME FOR JOTAKEY
		ing for the wrong site. That is inherent to every security key without a display.
Malicious USB/BLE peer	Sends malformed, oversized or out-of-order frames.	**Defeated.** Total parsers, bounded buffers, strict canonical CBOR, channel isolation (invariants 4–5). Verified by fuzzing and Lean proofs.
Network eavesdropper	Sees all WebAuthn traffic between browser and website.	**Irrelevant.** Only public keys and signatures over fresh challenges travel.
Database breach at the RP	Steals every stored public key and credential ID.	**Irrelevant.** Public keys cannot be used to log in; credential IDs are useless without the device's root key.
Thief, casual	Holds the device for minutes, no lab.	**Partially defeated.** No PIN in the MVP: the thief can log in where the user only needs a security key. Accounts that also require a password stay safe. (Adding a PIN is in the roadmap, §11.)
Thief, laboratory	Decapsulation, voltage/EM fault injection, power analysis.	**Jotakey key: partially defeated.** The root secret and the counter stay in the OPTIGA secure element (EAL6+ hardware). A thief who breaks the STM32 can **use** the key while holding it, and copy a credential only if they also obtain its ID from the website; they cannot copy the key as a whole (§7.5). **ESP32-C6: not defeated.** Published attacks bypass secure boot and flash encryption on this chip family (§7.4).
Supply chain	Tampered board or toolchain before provisioning.	**Out of scope** for the MVP; partially mitigated by reproducible builds and secure boot.

THE HONEST SUMMARY

Jotakey is a strong defence against **remote** attackers: phishing, malware and network attackers, which are by far the most common. Against a well-equipped attacker with physical possession, the two platforms differ. The jotakey key keeps its root secret in a certified secure element, so a stolen key cannot be cloned as a whole, although an attacker who breaks its microcontroller can use it while they hold it. The ESP32-C6 has no secure element and gives its secrets up. We say so plainly in the threat model, in `AGENTS.md`, and in the roadmap.

1.4 SECURITY INVARIANTS

The design is anchored by nine invariants. They are written in `AGENTS.md` (normative for anyone changing the code). Each is enforced in code and checked by tests. Where marked ⊢, it is also proved in Lean. Appendix A traces each invariant to its implementation, tests and theorems.

#	INVARIANT	LEAN
1	No signature carrying the UP flag without a user-presence event consumed within the same command.	⊢
2	The signature counter strictly increases, across power loss.	⊢
3	A credential ID unwraps only for the relying party it was minted for.	⊢
4	Parsers are total and bounded: every input yields a result, never a crash.	⊢
5	CTAPHID channels are isolated from each other.	⊢
6	Reset only within 10 s of power-up, and only with a touch.	⊢
7	Fail closed: failed self tests disable the authenticator permanently.	⊢
8	The root secret never leaves its hardware key store (OPTIGA or eFuse HMAC); secrets are zeroized, never logged.	
9	No secret-dependent branches or memory accesses in our code.	

02 CRYPTOGRAPHY IN ONE CHAPTER

FIDO2 uses a handful of standard primitives. This chapter explains just enough of each to follow the rest of the report. No mathematics beyond arithmetic is required.

2.1 HASH FUNCTIONS

A **hash function** such as SHA-256 maps any input to a fixed 32-byte **digest**. It is easy to compute and, as far as anyone knows, impossible to run backwards. It is also infeasible to find two inputs with the same digest (a **collision**). Hashes are used as compact, tamper-evident fingerprints. WebAuthn hashes the RP ID (`rpIdHash`) and the client data (`clientDataHash`), and the authenticator signs those hashes, not the raw data.

2.2 MESSAGE AUTHENTICATION CODES

A **MAC** is a hash with a secret key: `HMAC-SHA256(key, message)` produces a tag that only someone holding `key` can compute. Changing a single bit of the message changes the tag unpredictably. Jotakey uses HMAC in two ways:

- to **derive** independent keys from one root key, with a different label for each purpose (`"jotakey/mac/v1"`, `"jotakey/seed/v1"`);
- to **authenticate** credential IDs, so the device can recognise its own IDs and reject forged or foreign ones (§6.4).

HKDF (RFC 5869) is a standard construction built from HMAC. It turns key material plus a salt and a context string into as many independent-looking bytes as needed. Jotakey uses it to derive each credential's private key.

2.3 DIGITAL SIGNATURES: ECDSA ON P-256

A **signature scheme** has a private key, which signs, and a public key, which verifies. Anyone can check a signature, but only the private-key holder can produce one. WebAuthn's default algorithm, **ES256**, is ECDSA over the NIST P-256 elliptic curve with SHA-256.

- A **private key** is a random number d with $1 \le d < n$, where $n \approx 2^{256}$ is the order of the curve.
- The **public key** is the point $Q = d \cdot G$: the fixed generator point G "added to itself" d times. Computing Q from d is fast. Recovering d from Q is the elliptic curve discrete logarithm problem, believed infeasible.
- To **sign** a hash z, pick a secret one-time number k (the **nonce**), compute the point $k \cdot G$, and from it two numbers (r, s). Verification checks an equation using only Q.

THE NONCE IS EVERYTHING

If two signatures ever share the same k, or if k is even slightly predictable, simple algebra recovers the private key. This is how Sony's PlayStation 3 signing key leaked in 2010. Jotakey therefore uses **hedged** nonces (RFC 6979 §3.6): k is derived deterministically from the private key and the message, **and** mixed with 32 fresh bytes from the hardware random number generator. A broken RNG cannot cause nonce reuse, because the deterministic part protects. A fault attack that skips or repeats part of the computation cannot exploit determinism, because the random part protects. The known-answer self test checks the deterministic path against the RFC's published vector at every boot.

2.4 RANDOMNESS

Keys and nonces need **unpredictable** bits. The ESP32-C6 has a hardware true random number generator (TRNG) fed by thermal noise in the SAR ADC and in the radio. Hardware noise sources can fail: a stuck bit, a broken oscillator, a fault-injection glitch. NIST SP 800-90B specifies two cheap **health tests** that catch such failures continuously:

- **Repetition count test.** The same byte repeated too many times in a row (6, for an assumed 4 bits of entropy per byte) means the source is stuck.
- **Adaptive proportion test.** In each window of 512 bytes, the first byte must not recur 62 or more times. This catches a source that has become biased.

The cutoffs are computed exactly from the binomial distribution for a false-alarm rate of 2^{-20} (see `selftest.rs`). Jotakey runs both tests over 1024 bytes at boot and on every byte it uses afterwards. Any failure permanently disables the authenticator (invariant 7).

2.5 CONSTANT TIME

An attacker who can measure **how long** an operation takes, or how much power it draws, can learn secrets if the code's behaviour depends on them. The classic bug is comparing a secret tag with `==`, which stops at the first differing byte and so leaks how many bytes matched. Jotakey compares tags with `subtle::ConstantTimeEq` and uses the RustCrypto `p256` crate, whose scalar arithmetic is written to be constant-time. Its own code never branches on secret data (invariant 9).

03 FROM PASSWORDS TO PASSKEYS

3.1 WHAT IS WRONG WITH PASSWORDS

A password is a **shared secret**: the user knows it and the website stores something derived from it. Every login sends the secret, or something equivalent to it, to whoever is on the other end. That design has three structural flaws:

- **Phishing.** The user cannot reliably tell `example.com` from `examp1e.com`. Whatever they type on the fake site, the attacker replays on the real one. One-time codes from an app or SMS do not help: the fake site relays them in real time.
- **Reuse.** A breach at one site exposes the password everywhere it was reused.
- **Server breaches.** The site's database is a treasure chest of password hashes, which attackers crack offline.

3.2 PUBLIC-KEY LOGIN

Public-key authentication removes the shared secret. At registration, a fresh key pair is created **for that one site**. The site stores the public key. At login, the site sends a random **challenge**, and the user's device signs it with the private key. The private key never leaves the device, and the site stores nothing worth stealing.

That alone does not stop phishing: a fake site could relay the challenge too. FIDO adds the missing piece. The **browser**, which reliably knows which site it is talking to, includes the site's origin in what is signed. The **authenticator** only uses a credential for the site it was created for. A signature obtained on `examp1e.com`

names `examp1e.com`, and `example.com` rejects it. Phishing stops working, not because users get better at spotting fakes, but because the protocol never asks them to.

> **THE USER IS NO LONGER THE LAST LINE OF DEFENCE AGAINST PHISHING. THE PROTOCOL IS.**

3.3 A SHORT HISTORY

YEAR	MILESTONE
2012	The **FIDO Alliance** is founded (PayPal, Lenovo, Nok Nok Labs, Validity, Infineon, Agnitio; Google and Yubico join in 2013).
2014	**U2F** (Universal 2nd Factor): a USB key as a second factor next to the password. Google deploys it internally; in 2018 it reports that no employee account had been phished since security keys became mandatory in early 2017.
2018	**FIDO2** = W3C **WebAuthn** (browser API, Recommendation in 2019) + **CTAP2** (Client-to-Authenticator Protocol). Adds resident (discoverable) credentials, user verification by PIN or biometrics, and passwordless login.
2021	**CTAP 2.1**: credential management, PIN protocol v2, hmac-secret, enterprise attestation.
2022	Apple, Google and Microsoft commit to **passkeys**: FIDO credentials synchronised across a user's devices by the platform.
2023–25	**CTAP 2.2**: the **hybrid** transport (scan a QR code, phone acts as authenticator over BLE plus a cloud relay); passkeys become the default sign-in option on major sites.

3.4 PASSKEYS, PRECISELY

"Passkey" is the user-facing name for a **discoverable FIDO credential**. The word covers two deployment models with very different security properties:

	SYNCED PASSKEY	DEVICE-BOUND PASSKEY
Where the key lives	In the platform's cloud keychain (iCloud Keychain, Google Password Manager, 1Password …), end-to-end encrypted, copied to all the user's devices.	In one hardware authenticator (a security key or a TPM), never copied.
Survives losing the device	Yes	No; register a backup key.
Trust anchor	The platform account and its recovery process.	Possession of the physical device.
Typical use	Consumer accounts	High-assurance accounts, administrators, regulated environments.

Jotakey is a **device-bound** authenticator. The MVP implements **non-discoverable** credentials: the website must name the credential (send its ID) at login. This is how security keys work as a second factor, and how many enterprise deployments use them. **Discoverable** credentials, where the device stores the user handle so the website does not need to send an ID ("username-less" passkeys), are the first item on the roadmap (§11). Their implementation is laid out step by step there.

3.5 THE THREE PARTIES

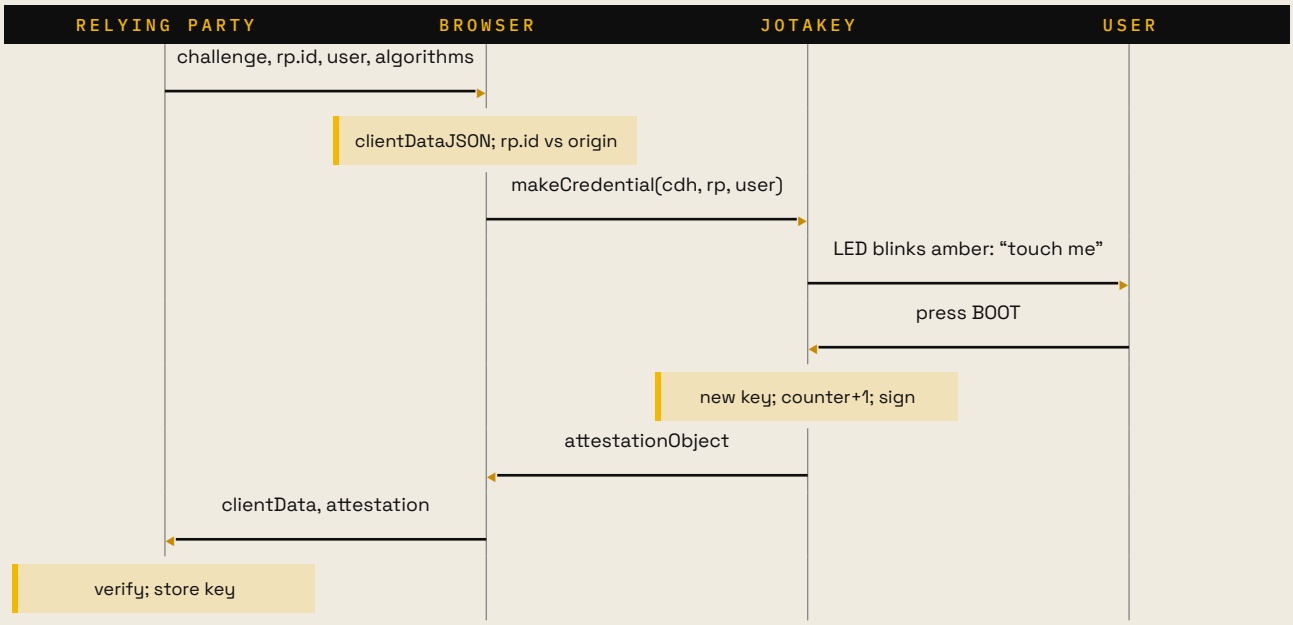
RELYING PARTY (RP)	The website, e.g. `example.com`. Stores public keys and credential IDs, issues challenges, verifies signatures. Identified by its **RP ID**, a domain.
CLIENT / PLATFORM	The browser plus the operating system. Knows the true origin of the page, builds `clientDataJSON`, talks to authenticators over USB, NFC, BLE or hybrid. Speaks WebAuthn upward and CTAP downward.
AUTHENTICATOR	The key: Jotakey. Holds or derives private keys, checks user presence, signs. Trusts nothing it receives.

THE ROLES IN EVERY FIDO CEREMONY

04 WEBAUTHN: THE TWO CEREMONIES

WebAuthn defines two **ceremonies** (the spec's word for a protocol that includes a human): **registration** creates a credential, **authentication** uses it. Both follow the same pattern: the RP sends a challenge, the browser wraps it, the authenticator signs, and the RP verifies.

4.1 REGISTRATION



REGISTRATION (NAVIGATOR.CREDENTIALS.CREATE)

The browser computes `clientDataJSON`, a small JSON document that records the operation type (`webauthn.create`), the RP's challenge and the **origin** of the page:

CLIENTDATAJSON (BUILT BY THE BROWSER, NEVER BY THE WEBSITE)

```
{"type":"webauthn.create","challenge":"q5sJ…7w","origin":"https://example.com","crossOrigin":false}
```

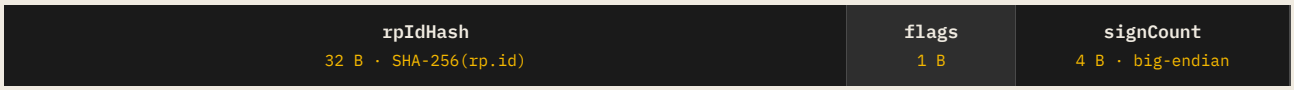
The authenticator never sees this JSON, only its SHA-256, the `clientDataHash`. It signs over that hash together with its own `authenticatorData`. The RP later checks that the challenge is the one it issued and the origin is its own. A phishing page at `examp1e.com` can relay the challenge, but cannot make the browser write a different origin.

4.2 THE RP ID AND WHY PHISHING FAILS

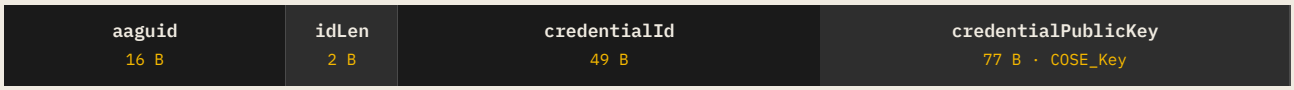
Every credential is created for an **RP ID**, a domain such as `example.com`. The browser only lets a page use an RP ID equal to its own host, or a registrable suffix of it: `login.example.com` may use `example.com`, `examp1e.com` may not. The authenticator hashes the RP ID into the credential (Jotakey binds it into the credential ID's authentication tag, §6.4) and refuses to use the credential for any other RP ID. So there are two independent checks: the browser checks origin against RP ID, and the key checks RP ID against credential.

4.3 AUTHENTICATORDATA

Both ceremonies return the same binary structure, which the authenticator builds and signs:



AUTHENTICATORDATA, ASSERTION FORM: 37 BYTES

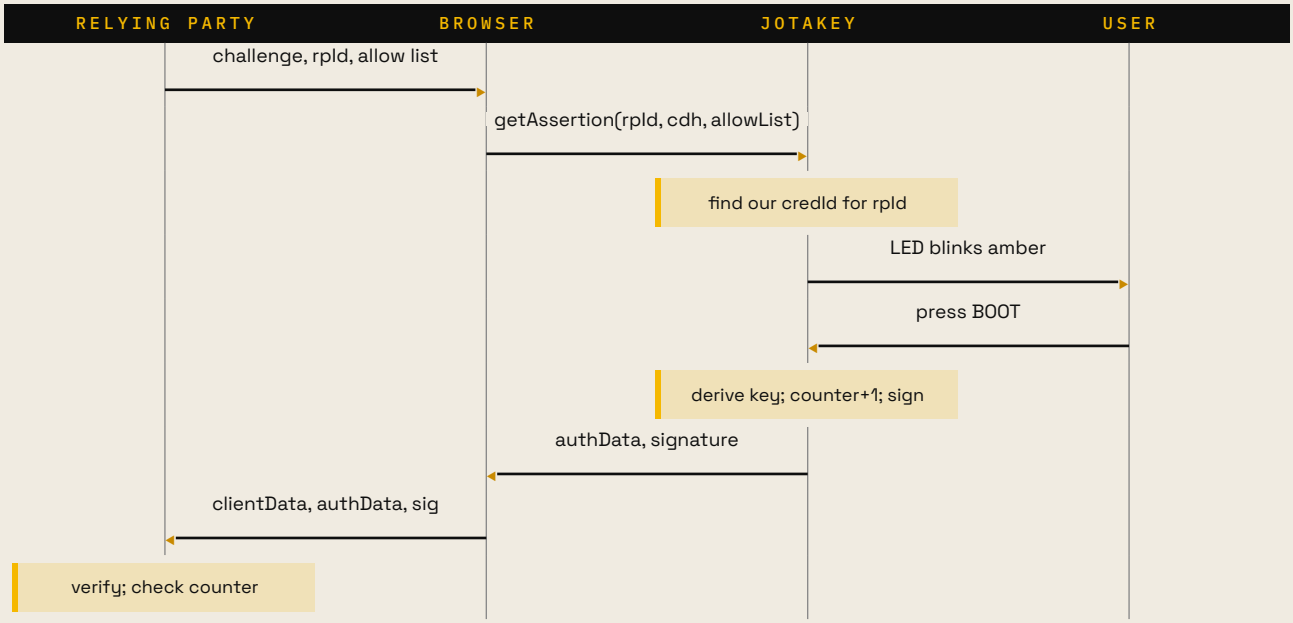


…PLUS ATTESTEDCREDENTIALDATA AT REGISTRATION (JOTAKEY: 181 BYTES IN TOTAL)

BIT	FLAG	MEANING
0	UP	User Present: someone physically interacted with the authenticator. **Jotakey always sets it** (invariant 1).
2	UV	User Verified: PIN or biometric checked. Not supported in the MVP.
3–4	BE/BS	Backup eligible / backed up (synced passkeys). Always 0 for Jotakey.
6	AT	Attested credential data included (registration only).
7	ED	Extension data included. Not used.

The **signature counter** lets an RP detect a cloned authenticator. If an assertion arrives with a counter not larger than the last one seen, two devices may share a key. Jotakey's counter is global, persisted to flash **before** any signature carrying it is released, and proved monotonic in Lean (§9.6).

4.4 AUTHENTICATION



AUTHENTICATION (NAVIGATOR.CREDENTIALS.GET)

The signature covers `authenticatorData ‖ clientDataHash`. The RP verifies it with the public key it stored at registration, checks the UP flag, the RP ID hash, the challenge, the origin, and that the counter increased.

4.5 ATTESTATION

At registration the authenticator can also prove **what kind of device** it is, by signing with a per-model **attestation key** whose certificate chains to the manufacturer. Formats include `packed`, `tpm`, `android-key`, `fido-u2f`, `apple` and `none`. Attestation serves enterprises that must allow only certified models. It costs privacy (keys of one batch are linkable) and requires a certificate authority.

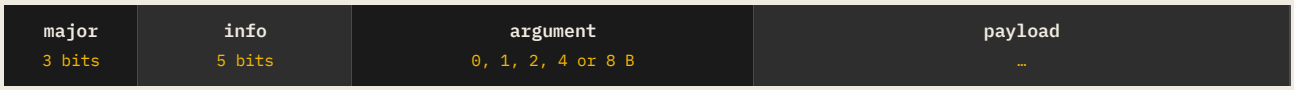
Jotakey uses **packed self-attestation**: the attestation statement is signed with the new credential's own private key. This proves possession of the key and nothing about the device. Most consumer RPs request no attestation (`"none"`) anyway.

05 CTAP2: TALKING TO THE AUTHENTICATOR

WebAuthn is the browser's API. **CTAP**, the Client-to-Authenticator Protocol, is what the browser actually sends to the key. CTAP2 messages are a command byte followed by a **CBOR** map. The answer is a status byte followed by a CBOR map.

5.1 CBOR, AND WHY STRICTNESS MATTERS

CBOR (RFC 8949) is a binary JSON: integers, byte strings, text strings, arrays, maps, booleans. Each item starts with a **head** byte: 3 bits of major type and 5 bits of "additional information". The latter holds a small value directly (0–23) or announces 1, 2, 4 or 8 following bytes.



A CBOR HEAD: MAJOR TYPE 3 (TEXT), LENGTH 11 IN THE LOW BITS, THEN 11 BYTES OF UTF-8

The same value can often be written in several ways: `24` as `0x18 0x18`, as `0x19 0x00 0x18`, and so on. Map keys can appear in any order or twice. For a parser that guards keys, this freedom is dangerous. If the authenticator and some other component disagree about which of two duplicate `rpId` keys counts, an attacker can show one value to one component and another to the other. CTAP2 therefore requires **canonical** CBOR. Jotakey's decoder rejects everything else:

- every integer, length and count in its shortest form;
- definite lengths only;
- map keys strictly increasing in canonical order (shorter encoding first, then bytewise), which also rules out duplicates;
- no tags, floats or `undefined`; nesting depth at most 4.

The Lean model of this decoder carries two theorems (§9.5). **Round trip**: decoding an encoded head returns exactly the encoded value. **Uniqueness**: anything the decoder accepts is exactly the canonical encoding of what it decoded. A differential test checks that the Rust decoder and the Lean model agree on over 100 000 inputs.

5.2 COMMANDS

BYTE	COMMAND	JOTAKEY MVP
0x01	authenticatorMakeCredential	Yes. ES256 only; `rk` and `uv` options refused.
0x02	authenticatorGetAssertion	Yes. Allow list required (non-resident credentials).
0x04	authenticatorGetInfo	Yes. Versions, AAGUID, options, limits.
0x06	authenticatorClientPIN	No (roadmap). Zero-length `pinAuth` probes get `PIN_NOT_SET` after a touch.

BYTE	COMMAND	JOTAKEY MVP
0x07	authenticatorReset	Yes. Within 10 s of power-up, with a touch.
0x08	authenticatorGetNextAssertion	Not needed without resident keys.

AUTHENTICATORGETINFO RESPONSE, AS JOTAKEY SENDS IT (DIAGNOSTIC NOTATION)

```
{ 1: ["FIDO_2_0"],                        ; versions
  3: h'6a6f74616b65792d7630312d63360001',  ; AAGUID
  4: {"rk": false, "up": true, "plat": false},
  5: 1200,                                 ; maxMsgSize
  7: 8,                                    ; maxCredentialCountInList
  8: 49 }                                  ; maxCredentialIdLength
```

CTAP allows `getAssertion` with option `up: false`, a **silent** assertion that produces a valid signature without a touch. Clients use it as a **pre-flight**: before asking the user to touch, they find out which credential in their list the key recognises. python-fido2 and browsers rely on it, and an early Jotakey that refused it could register but never sign in. Jotakey therefore answers it, signing at once with the UP flag **clear** in the authenticator data. WebAuthn requires relying parties to reject an assertion without UP, so a silent assertion cannot log anyone in. Every signature that carries UP still needs a fresh touch (invariant 1), and every signature, silent or not, moves the counter.

5.3 CTAPHID: CTAP OVER USB

USB security keys are HID devices, like keyboards. Each report is exactly 64 bytes, so longer messages are split into an **initialization packet** and **continuation packets**:



INITIALIZATION PACKET (57 PAYLOAD BYTES)



CONTINUATION PACKET (59 PAYLOAD BYTES)

Several applications may talk to the key at once, so CTAPHID multiplexes **channels**, identified by a 4-byte CID that `CTAPHID_INIT` allocates. Only one transaction runs at a time. A packet from another channel during a transaction is answered with `ERR_CHANNEL_BUSY` and must not disturb the transaction in progress. This is **channel isolation** (invariant 5), proved for the Lean model in §9.6. While waiting for the user, the device sends a `KEEPALIVE` with status `UPNEEDED` at least every 100 ms, so the browser can show “touch your security key”. `CTAPHID_CANCEL` aborts the wait.

5.4 FIDO OVER BLE

Over Bluetooth Low Energy, the authenticator exposes a GATT service with UUID `0xFFFD` and four characteristics. The host writes request fragments to `fidoControlPoint`. The device answers with notifications on `fidoStatus`. `fidoControlPointLength` sets the fragment size, and `fidoServiceRevisionBitfield` advertises FIDO2 support. Framing works like CTAPHID without channels: an initial fragment `CMD ‖ HLEN ‖ LLEN ‖ DATA`, then `SEQ ‖ DATA`. The link must be encrypted (LE Secure Connections) before the FIDO characteristics accept anything.

06 JOTAKEY'S ARCHITECTURE

6.1 THE ESP32-C6 BOARD, AND THE USB PROBLEM

This section concerns the ESP32-C6 DevKit only. The jotakey key's STM32L432 has a general-purpose USB device controller, so the key enumerates as an ordinary FIDO HID device and needs no bridge, on any operating system.

The ESP32-C6 DevKit has two USB-C ports. The one labelled **UART** goes through a USB-to-serial bridge chip to the C6's UART0. The one labelled **USB** is wired to the C6's own USB peripheral. On the C6, that peripheral is the **USB-Serial/JTAG controller**, a fixed-function block implemented entirely in hardware. It can be a serial port and a JTAG debug probe, and nothing else. In particular, it **cannot enumerate as a USB HID device**. Every USB security key is a HID device. (Chips such as the ESP32-S3 have a general USB OTG controller. The C6 does not.)

Jotakey therefore splits USB into two halves. The device speaks the exact CTAPHID protocol, with the same 64-byte reports, but sends them over the serial port inside a small framing layer. On the host, a tiny Linux daemon, `jotakey-bridge`, creates a **virtual** HID device through the kernel's `/dev/uhid` interface and copies reports in both directions. Browsers see an ordinary FIDO security key.

BROWSER / PYTHON-FIDO2	Opens `/dev/hidrawN`, speaks CTAPHID, unaware of the bridge.
LINUX HID CORE	Virtual device created via `/dev/uhid` with the FIDO usage page (0xF1D0).
JOTAKEY-BRIDGE	≈330 lines, std Rust: uhid ⇄ serial, COBS + CRC-32 framing. **Outside the security boundary.**
USB-SERIAL/JTAG	The DevKit's "USB" port: fixed-function serial channel.
FIRMWARE: USB TASK	Deframer → CtapHid → authenticator, and back.

THE USB PATH, TOP TO BOTTOM

> **WHY THE BRIDGE DOES NOT WEAKEN SECURITY**
> FIDO already treats the host as hostile: the authenticator trusts nothing it receives, and every signature needs a touch on the device itself. The bridge runs **on** the host, so it can do nothing that host malware could not do anyway. It never sees key material, only the same CTAP messages any USB host sees. The cost is convenience: the bridge is Linux-only. The roadmap (§11) lists a small USB-capable co-processor as the path to a standalone, OS-independent device.

The framing on the serial line is deliberately simple:

COBS-encoded body (no zero bytes) 64-byte report + 4-byte CRC-32/ISO-HDLC, ≤ 70 B	0x00 delimiter

ONE FRAME ON THE SERIAL LINE: COBS(REPORT ‖ CRC-32) FOLLOWED BY 0X00

COBS (Consistent Overhead Byte Stuffing) removes every zero byte from the body, so a zero **always** marks the end of a frame. After line noise or a half-written frame, the receiver resynchronises at the next zero. The CRC catches corruption. It is an integrity check, not a security mechanism. Corrupt frames are silently dropped, and CTAPHID's own timeouts take over. Logs never mix with frames: the firmware prints only to the **UART** port. That gives the two ports clean roles: "USB" carries the authenticator, "UART" carries diagnostics. §6.7 describes the bridge and the framing in full detail.

6.2 SOFTWARE LAYERS

JOTAKEY-CORE	`no_std`, no allocation, `#![forbid(unsafe_code)]`, no `unwrap` / `panic!` / unchecked indexing or arithmetic (enforced by Clippy). CBOR, CTAP2 commands, CTAPHID, BLE framing, serial framing, credential wrapping, power-safe storage, self tests. **All** bytes from the host are interpreted here.
JOTAKEY-FW	ESP32-C6 firmware (esp-hal, embassy, trouBLE). Thin adapters: TRNG, eFuse HMAC key, flash, button, LED, USB-Serial/JTAG, BLE GATT. Implements the core's `Platform` trait.
JOTAKEY-TESTFW	On-device test firmware (embedded-test + probe-rs).
JOTAKEY-BRIDGE	Linux host daemon (serial ⇄ uhid).
FORMAL/	Lean 4 models, proofs, and the executable CBOR specification.

CRATES AND THEIR TRUST LEVELS

The core is written so that it does not know it runs on a microcontroller. Hardware is reached only through one trait:

CRATES/JOTAKEY-CORE/SRC/PLATFORM.RS (ABRIDGED)

```rust
pub trait Platform {
    fn random_bytes(&mut self, out: &mut [u8]) -> Result<(), PlatformError>;
    fn derive_secret(&mut self, label: SecretLabel, out: &mut [u8; 32]) -> Result<(), PlatformError>;
    fn increment_counter(&mut self) -> Result<u32, PlatformError>;   // persisted before return
    fn rotate_epoch(&mut self) -> Result<(), PlatformError>;
    fn uptime_ms(&self) -> u64;
}
```

User presence is deliberately **not** in the trait. Waiting for a button is asynchronous and involves keepalives and cancellation, and none of that belongs inside the protocol logic. Instead, the core is a **resumable state machine**. `Authenticator::process` validates a request completely. If it needs a touch, it parks the validated parameters in a `Pending` value and returns `NeedsUserPresence`. The firmware blinks the LED, sends keepalives, watches the button and the cancel signal, then calls `Authenticator::complete` with `Confirmed`, `Timeout` or `Cancelled`. Signing code is reachable **only** from `complete` under `Confirmed`. That structural fact is what the Lean model formalises (§9.6).

6.3 THE KEY HIERARCHY

ROOT (HARDWARE KEY STORE)	The device's one long-term secret. Software can only ask the hardware to **use** it, never read it: the OPTIGA secure element on the key, the eFuse HMAC peripheral on the C6.
EPOCH (FLASH)	32 random bytes, replaced by authenticatorReset. Makes reset possible while the root stays fixed for the device's life.
TAG, SEED (PER CREDENTIAL)	F(root, "jotakey/tag/v1" ‖ epoch ‖ 0x01 ‖ rpIdHash ‖ nonce), F(root, "jotakey/seed/v1" ‖ epoch ‖ rpIdHash ‖ nonce), computed by the hardware for one credential at a time. Zeroized after use.
CREDENTIAL KEY D	HKDF-SHA256(seed, salt = rpIdHash, info = "jotakey/cred/v1" ‖ nonce ‖ ctr) → valid P-256 scalar.

FROM THE HARDWARE ROOT SECRET TO A CREDENTIAL'S PRIVATE KEY

The hardware computes each credential's values from the root directly. The microcontroller therefore never holds a secret that covers more than one credential. That matters on the day an attacker manages to run their own code on it (§7.5). They can then ask the hardware for the values of any credential whose ID they hold, and so use or copy that credential. But there is no key to take away that would let them compute every credential, or mint new ones, on their own computer afterwards.

The two platforms differ only in where the root lives and in the function `F`:

	JOTAKEY KEY	ESP32-C6
Root	`K_root`: 64 bytes in OPTIGA data object `0xF1D0`, type pre-shared secret. Created on the device at provisioning from both chips' random generators, then locked: never readable, never changeable, usable only over the encrypted link (§7.5).	`K_dev`: 256 bits in eFuse BLOCK_KEY0, purpose HMAC_UP, read-protected by hardware.
`F`	TLS 1.2 PRF with SHA-256, run by the OPTIGA's `DeriveKey` command. The Trust M V1 has no HMAC command; its PRF is built from HMAC-SHA256.	HMAC-SHA256 in the HMAC peripheral.
Epoch	STM32 flash, behind readout protection.	Encrypted flash.
Counter	OPTIGA hardware up-counter.	Flash record log (§6.5).

The credential format and its proofs (§9.6) are the same on both platforms: the proofs treat the tag function as a parameter, and say nothing about where it is computed.

> **IN THE CODE**
> `Platform::derive_secret(label, context)` returns `F(root, label ‖ epoch ‖ context)`, and the core only ever passes one credential's context. An earlier version derived two epoch-wide keys, `K_mac` and `K_seed`, and computed every tag and HKDF on the microcontroller: exactly the "key to take away" described above. If the hardware fails while the core checks an ID, the request fails; it never reads as "not our credential".

Nothing secret is ever **stored** in the microcontroller's flash except the epoch (and, on the key, the secret that encrypts the link to the OPTIGA). The epoch is useless without the root, which the flash never holds.

6.4 CREDENTIAL IDS: KEYS BY DERIVATION

For a non-resident credential the website stores the credential ID and hands it back at login. Jotakey makes the ID carry everything it needs to **re-derive** the private key, and authenticates it so that it cannot be forged:



JOTAKEY CREDENTIAL ID: 49 BYTES

At `getAssertion`, Jotakey recomputes the tag for the **requesting** RP and compares it in constant time. An ID minted for `example.com` presented by `examp1e.com` fails the check, because the RP ID hash is inside the tag (invariant 3). A forged or foreign ID fails too. The nonce is not secret, and the private key is never in the ID, not even encrypted.

> **PROVED IN LEAN (§9.6)**
> `unwrap_accepts_only_wrapped_ids`: anything `unwrap` accepts is **exactly** `wrap(rp, nonce)` for the recovered nonce. `credential_bound_to_relying_party`: an ID minted for one RP never unwraps at another, **assuming** the MAC does not collide across RPs. That assumption is the cryptographic part. It holds for HMAC-SHA256 and for the TLS PRF built on it, truncated to 128 bits, except with probability $\approx 2^{-128}$, and it is stated explicitly as a hypothesis of the theorem rather than hidden.

6.5 PERSISTENT STATE

On the jotakey key, the signature counter lives in the OPTIGA instead (§7.5): a hardware up-counter that can only increase, survives power loss by construction, and cannot be rolled back even by firmware an attacker controls. The record log described here holds the counter and the epoch on the C6, and the epoch on the key.

On the C6, only two values persist: the signature counter and the epoch. Each lives in its own power-loss-safe **record log** of two flash sectors. A new value is appended as a record `MAGIC ‖ seq ‖ payload ‖ CRC-32`

after the last one. When a sector fills, the **other** sector is erased and the record goes there. The sector holding the current value is never erased.

The append operation reads the record back and only reports success once it verifies. The authenticator only releases a signature carrying counter $N + 1$ after that success. If power fails mid-write, the torn record is ignored at the next boot. That is harmless, because nothing carrying $N + 1$ was ever released. A property test cuts power at random byte offsets (including in the middle of a sector erase) across thousands of runs and checks that no confirmed counter value is ever lost or handed out twice. The test firmware repeats the torn-write check on real NOR flash.

6.6 USER EXPERIENCE

LED	MEANING
off	Idle, healthy.
amber blink	A website is asking: touch BOOT to approve. Times out after 30 s.
blue blink	CTAPHID_WINK: the platform asks the device to identify itself.
solid red	Self test or hardware fault. The authenticator refuses everything until reset; the UART log says why.
magenta at boot	**INSECURE** development build (`dev-key` feature, root key derived from the MAC address).

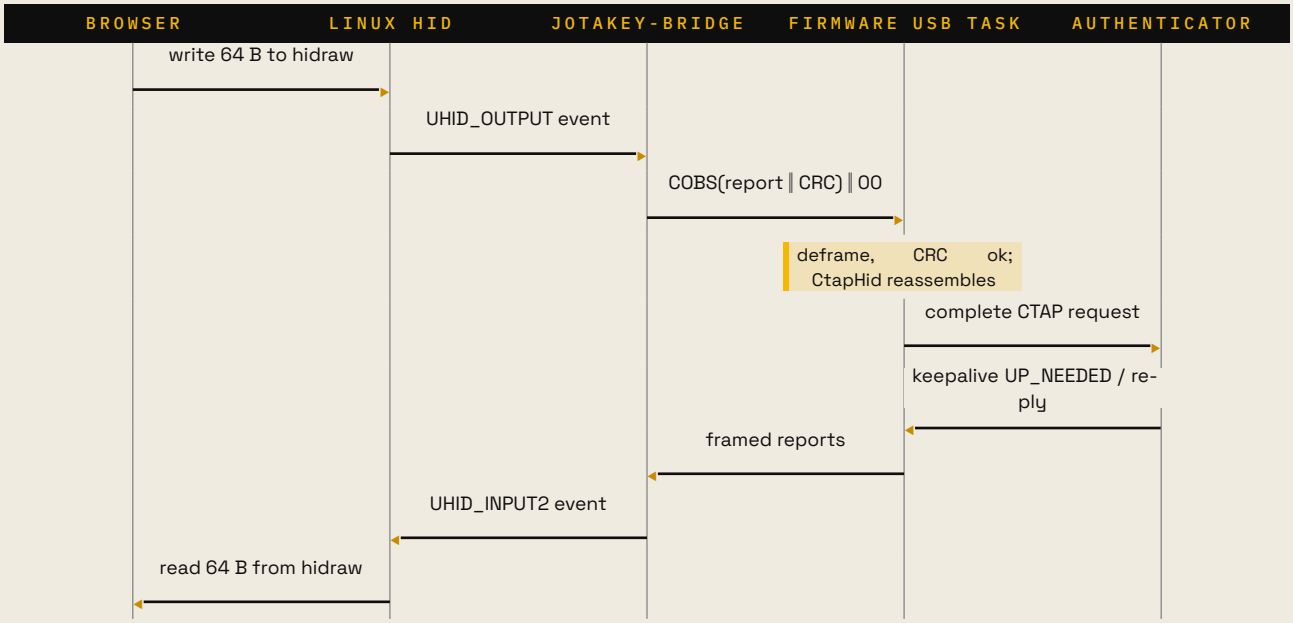
A touch counts only if the button is seen **released** after the request arrived, then held for two consecutive 20 ms polls. A button taped down, or still held from a previous request, never approves anything. Holding BOOT for 3 s while idle turns on BLE for two minutes, so the radio is never exposed without the owner's intent.

6.7 THE USB BRIDGE IN DETAIL

§6.1 introduced `jotakey-bridge` in one paragraph. This section describes it completely: what happens to a request on its way from the browser to the chip, what each layer does, what goes wrong and how it recovers, and what a hostile bridge could and could not do. The technique is general. Any microcontroller with a serial port but no USB device controller can appear to Linux as a HID device this way, which makes it useful well beyond Jotakey.

The guiding rule is that the bridge **understands nothing**. It does not parse CTAPHID, CBOR or CTAP. It moves opaque 64-byte reports between two places and checks only that they arrived intact. Every protocol decision is made on the device, by the same core code that is tested, fuzzed and modelled in Lean. The bridge is about 330 lines in total: `main.rs` (127) and `uhid.rs` (202, including tests).

The journey of one request



A REQUEST AND ITS ANSWER. MESSAGES LARGER THAN ONE REPORT REPEAT THE TOP HALF PER PACKET.

The browser cannot tell this apart from a USB key. It opens `/dev/hidrawN` and writes 64-byte output reports. The kernel hands each one to the bridge as a `UHID_OUTPUT` event. The bridge wraps the report in a frame and writes it to the serial port. On the chip, the usb task deframes it, checks the CRC, and feeds the report to the `CtapHid` state machine, which reassembles multi-packet messages. A complete request goes to the authenticator task. Replies and keepalives travel the same path backwards: `CtapHid` splits them into reports, the usb task frames them, the bridge unframes them and injects each one into the kernel with `UHID_INPUT2`, and the browser reads it from hidraw.

On the host: a virtual HID device through uhid

`uhid` is a Linux kernel interface for implementing HID devices in user space. A program opens `/dev/uhid` and exchanges fixed-size `struct uhid_event` records with the kernel. To the rest of the system, the result is a normal HID device. It gets a hidraw node, udev sees it, and browsers enumerate it. The bridge uses only a handful of events:

EVENT	DIRECTION	USE IN THE BRIDGE
`UHID_CREATE2`	write	Once at start: name “Jotakey FIDO2 (bridge)”, bus USB, VID:PID `1209:0001`, and the report descriptor below.
`UHID_OUTPUT`	read	The host wrote a report. Forwarded to the serial port.
`UHID_INPUT2`	write	Deliver a report from the device to the host.
`UHID_GET_REPORT`, `UHID_SET_REPORT`	read	Feature-report requests, which FIDO does not use. Answered at once with `EIO`, because the kernel waits for a reply.
`UHID_START`, `OPEN`, `CLOSE`, …	read	Notifications. Ignored.
`UHID_DESTROY`	write	On exit (Rust `Drop`), so the virtual key disappears with the bridge.

The **report descriptor** is what makes the device a FIDO key. It declares the FIDO Alliance usage page `0xF1D0`, usage `CTAPHID`, and one 64-byte input and one 64-byte output report, exactly as CTAP 2.0 §8.1.8.2 prescribes. Browsers and libraries look for that usage page, not for a vendor or product ID. systemd’s `fido_id` helper recognises the same descriptor and grants the logged-in user access to the hidraw node. The VID:PID pair `1209:0001` is the pid.codes test identifier, suitable for development but not for a product.

The bridge has its own uhid encoder (`uhid.rs`) instead of a crate. The obvious crate, `uhid-virt`, needs `libclang` at build time to generate bindings. The structures are packed, so the encoder is plain byte

arithmetic, with no `unsafe` and no C bindings. Unit tests pin the layout against the kernel's: an event is exactly 4 376 bytes, and the `CREATE2` fields sit at the right offsets. One subtlety: depending on the path a report takes through the kernel, an output report from a device without report IDs can arrive with a leading report-number byte of 0. The bridge accepts 64 or 65 bytes and strips that byte. Anything else is ignored.

On the wire: one frame, byte by byte

The serial link is a byte stream, so reports need boundaries. Each report travels as one frame: `COBS(report ‖ CRC-32(report)) ‖ 0x00`. This is the actual frame for the first packet every browser sends, a `CTAPHID_INIT` on the broadcast channel with an 8-byte nonce. It was produced by the core's `serial::encode` and cross-checked against an independent Python implementation:

```
CTAPHID_INIT REQUEST: 64-BYTE REPORT → 70-BYTE FRAME

report   ff ff ff ff 86 00 08 11 22 33 44 55 66 77 88 00 … 00     (49 zero bytes)
          └ CID ──────┘ cmd └len┘ └ nonce ─────────────────┘
CRC-32   a5 9f 94 f7 → appended little-endian: f7 94 9f a5

frame    06 ff ff ff ff 86                  5 data bytes, then a zero
         0a 08 11 22 33 44 55 66 77 88      9 data bytes, then a zero
         01 01 01 … 01                      48 × "no data, then a zero"
         05 f7 94 9f a5                     4 data bytes (the CRC), end of body
         00                                 delimiter
```

COBS (Consistent Overhead Byte Stuffing) removes every zero byte from the body. Each code byte `n` means "copy the next `n − 1` bytes, then output a zero", except at the very end. Because the body never contains `0x00`, a zero on the wire **always** means "end of frame". A receiver that starts listening mid-stream, or sees a corrupted byte, recovers at the next zero. The overhead is bounded: one code byte per 254 data bytes plus the delimiter, so a frame is at most 70 bytes for a 64-byte report.

The **CRC-32** (the ISO-HDLC polynomial used by zlib and Ethernet; check value `CBF43926`) detects corruption: noise, a dropped byte, or two frames merged by a lost delimiter. It is not a security mechanism. Anyone on the host can compute a valid CRC, and nothing here needs to keep the host out, because the device validates every message anyway.

The receiver, `serial::Deframer`, is a 70-byte buffer and a little state. It collects bytes until a zero. Empty frames, which are just idle delimiters, are ignored. A frame that overflows the buffer, fails COBS decoding, has the wrong length or fails the CRC is dropped and counted. Nothing is ever retransmitted. CTAPHID already has timeouts and retries, and duplicating them in the framing layer would only add states. A unit test feeds a log line, a corrupted frame and a good frame in one stream, and checks that exactly the good one comes out. The `serial_deframe` fuzz target has pushed millions of arbitrary byte streams through the same code.

Two properties of the C6's USB-Serial/JTAG port matter for the host side. The baud rate is meaningless, because the data moves at USB full speed whatever the port is told. More importantly, the DTR and RTS control lines are wired to the chip's reset logic (that is how flashing tools reset it), so opening the port the usual way can reboot the authenticator. The bridge opens the port with DTR off and clears RTS.

On the device: the usb task

On the chip, `crates/jotakey-fw/src/usb.rs` runs one task that waits on four things at once: bytes from the serial port, a reply from the authenticator, a keepalive request, and a 50 ms timer. Incoming bytes go through the `Deframer` into `CtapHid::handle_packet`. `CtapHid` implements channel allocation, reassembly, busy and cancel handling, and transaction timeouts. The timer drives those timeouts even when no bytes arrive. Each outgoing packet `CtapHid` produces is framed and written back.

This placement is deliberate. The whole CTAPHID state machine runs on the device, in the core, so channel isolation (invariant 5) is enforced by code the Lean model of §9.6 describes, not by the bridge. A bridge that misbehaved could not break isolation between two host applications. It could only feed the device packets, which any USB host can do.

The firmware has one authenticator and two transports. If a USB request arrives while a BLE request is being processed, the usb task answers `CTAP1_ERR_CHANNEL_BUSY` itself, and the host retries. The firmware never prints anything on this port. Logs go to the **UART** port only, so the CTAPHID stream contains nothing but frames.

Lifecycle and failure modes

The bridge creates the virtual device once, then loops: open the serial port, pump reports until the port hangs up or fails, wait 500 ms, try again.

SITUATION	WHAT HAPPENS
Board unplugged	The serial port errors and the bridge goes back to waiting. The virtual key **stays**, so the browser still sees a key, but its requests time out. Reports the host writes meanwhile queue in the kernel (uhid buffers a limited number of events) and may be delivered after the board returns.
Board reset or replugged	The bridge reconnects within about 500 ms. The firmware has forgotten all channels, so packets on old channel IDs get `ERR_INVALID_CHANNEL`. Host libraries then send a fresh `CTAPHID_INIT`.
Line noise or a half-written frame	The CRC or COBS check fails, the frame is dropped, the stream resynchronises at the next zero, and CTAPHID timeouts recover the transaction.
Bridge stopped	`UHID_DESTROY` removes the virtual key. If the process is killed outright, the kernel removes it when `/dev/uhid` is closed.
Wrong serial port	The bridge would send frames to some other device. Use the stable name under `/dev/serial/by-id/` (the entry for the Espressif USB JTAG/serial unit) rather than `/dev/ttyACM0`, whose number depends on plug order.
Two bridges on one port	Both read the same stream and each gets part of it. Frames break and are dropped. Run exactly one.

What a hostile bridge can and cannot do

The bridge is outside the security boundary, like the rest of the host. Treating it as an attacker gives this picture:

A MALICIOUS OR COMPROMISED BRIDGE CAN…	…BUT IT CANNOT
Drop, delay, reorder or replay any report (denial of service).	Obtain a signature without a touch on the device (invariant 1, proved in §9.6).
Send the device any request of its choosing, for any website.	Learn any private key or root key. Neither ever leaves the chip.
Read all CTAP traffic: relying-party ID hashes, credential IDs, and the user name and display name sent in makeCredential.	Use a credential for another website. IDs are bound to `rpIdHash` (invariant 3).
Pretend to be a security key itself, since it is software that creates HID devices.	Make the device skip its checks. Every message is parsed and validated by the core.

Everything in the left column is also available to any program with root on the host, or to a malicious USB host. That is the threat model FIDO is designed for. The bridge adds no new attack on the key. It is a new place for host-side malware to hide, which is one more reason the roadmap replaces it with a USB co-processor.

> **ACCESS TO /DEV/UHID IS POWERFUL**
> A process that can write to `/dev/uhid` can create **any** HID device, including a keyboard that types commands. `scripts/99-jotakey.rules` gives the `plugdev` group that access, which is convenient on a development machine. On a shared or long-lived system, run the bridge as a dedicated system user that alone has access to `/dev/uhid` and the serial port, for example as a systemd service, rather than widening a group most users belong to.

Running it

BRIDGE SETUP ON LINUX

```
sudo cp scripts/99-jotakey.rules /etc/udev/rules.d/ && sudo udevadm control --reload
sudo usermod -aG plugdev,dialout "$USER"     # uhid + serial access; log in again
                                              # (the serial group is uucp on Arch)
ls /dev/serial/by-id/                         # find the Espressif USB JTAG/serial entry
just bridge /dev/serial/by-id/usb-Espressif…  # or: cargo run -p jotakey-bridge --release -- <port>
# jotakey-bridge: virtual FIDO device created; waiting for …
# jotakey-bridge: connected to …
```

With the bridge running, `python3 scripts/fido2_check.py`, `just host-tests` and any browser on the machine see the key. Stop the bridge with Ctrl-C.

How it is tested, and current status

PART	EVIDENCE
uhid encoding	Unit tests pin the event size (4 376 bytes) and the `CREATE2` layout to the kernel's, and check `OUTPUT` decoding and the `EIO` replies.
Framing	Round trip of every possible report (property test), all-zero and zero-free reports, corruption and resynchronisation, and the `serial_deframe` fuzz target.
Pump loop	The real `pump` runs between a fake uhid (a datagram socket pair, which keeps event boundaries like `/dev/uhid`) and a pseudo-terminal standing in for the serial port. The test checks one report in each direction, that the idle bridge wakes at most twice in 300 ms (the earlier 5 ms polling loop woke 60 times), and that a hang-up ends the pump so the bridge can reconnect.
Whole path	`jotakey-host-tests` runs a simulated device wired exactly like the usb task (framing → `CtapHid` → authenticator), so every simulated request crosses the real framing code in both directions.
On hardware	**Not yet run.** No board was attached while it was written, and `/dev/uhid` needs the permissions above. `just host-tests` with a board attached is the first check to run.

The bridge sleeps in `poll(2)` on both file descriptors, the uhid device and the serial port, with no timeout. It wakes only when the host writes a report or the board sends bytes, forwards them at once, and uses no CPU while idle. It needs no timer of its own, because all CTAPHID timeouts run on the device. A hang-up or error on the serial port ends the loop, and the bridge goes back to waiting for the board.

One limitation is known. The bridge is **Linux-only**, because `uhid` is a Linux interface; other systems would need their own virtual-HID mechanism.

07 HARDWARE SECURITY

Software can be proved correct. Silicon can be attacked. This chapter describes what the ESP32-C6 offers, how Jotakey uses it, and where it falls short (§7.1–7.4). It then describes the jotakey key, which adds the secure element the C6 lacks (§7.5), and how its board is designed (§7.6).

7.1 WHAT THE CHIP PROVIDES

FEATURE	ROLE IN JOTAKEY
eFuse key blocks (6 × 256 bit)	One-time-programmable. `K_dev` is burned into BLOCK_KEY0 with purpose `HMAC_UP`. Keys with this purpose are read-protected: the CPU cannot read them, only the HMAC peripheral can use them.
HMAC peripheral	Computes HMAC-SHA256 with an eFuse key the CPU never sees. Jotakey's only access to `K_dev`.

FEATURE	ROLE IN JOTAKEY
Secure Boot v2	The ROM verifies the bootloader's RSA-3072 signature, and the bootloader verifies the app's, against a key digest in eFuse. Only firmware signed by the owner runs.
Flash encryption	XTS-AES encryption of flash contents with an on-chip key generated at first boot. Protects the firmware image and the `jotakey` partition (epoch, counter) against off-chip reading.
TRNG	Hardware entropy from SAR ADC and RF noise; health-tested continuously (§2.4).
JTAG / download-mode fuses	Burned at provisioning so a physical attacker cannot attach a debugger or dump flash through the ROM loader.

7.2 PROVISIONING

Provisioning is a one-way trip: eFuses cannot be unburned. `scripts/provision.sh` documents every step and **prints** the commands by default. It executes one step at a time, only when a human passes `--execute <step>` and types the chip's MAC address to confirm. `AGENTS.md` forbids automation from ever running it.

1. **Root key.** 32 random bytes generated in RAM (`/dev/shm`), burned with `espefuse.py burn_key BLOCK_KEY0 … HMAC_UP`, then shredded. The key then exists only inside the chip. A better future option is to generate it on the device, so it never touches the host.
2. **Secure Boot v2.** An RSA-3072 signing key kept offline. A signed bootloader built with ESP-IDF, and signed application images.
3. **Flash encryption, release mode.** The key is generated on-chip at first boot, and plaintext reflashing is disabled.
4. **Lockdown.** Disable pad and USB JTAG (the USB **serial** function stays, because it carries the CTAPHID tunnel). Switch ROM download mode to "secure download".

Until step 1 is done, the firmware refuses to operate: the HMAC peripheral rejects an unprovisioned key, the boot self check fails, and the LED goes red. For development, the `dev-key` cargo feature substitutes a software key derived from the MAC address. It is loudly announced at boot and must never hold real credentials.

7.3 SIDE CHANNELS

- **Timing.** All comparisons of secret data are constant-time. Scalar multiplication uses RustCrypto's constant-time implementation in software, rather than the C6's ECC accelerator, whose side-channel behaviour is undocumented.
- **Power and EM.** A microcontroller without dedicated countermeasures leaks information through its power draw. An attacker with an oscilloscope and physical access can, in principle, extract an ECDSA key from enough traces. Hedged nonces reduce, but do not remove, that risk.

7.4 FAULT INJECTION: THE LIMIT OF THIS CHIP

> **PUBLISHED ATTACK: ESPRESSIF ADVISORY AR2023-007**
> In January 2024 Espressif published **AR2023-007**, "Security Advisory Concerning Bypassing Secure Boot and Flash Encryption using CPA and FI attack on ESP32-C3 and ESP32-C6". It combines correlation power analysis with voltage-glitch fault injection. An attacker with physical access and lab equipment can run code before Secure Boot rejects it, and then extract device secrets. Earlier advisories (AR2023-005, CVE-2019-15894) describe similar attacks on the original ESP32. Treat a stolen Jotakey as compromised if the thief is well-resourced, and revoke its credentials at your relying parties.

This is why the threat model (§1.3) marks laboratory attackers as **not defeated**. The standard answer is a **secure element**: a small chip designed and certified (Common Criteria EAL5+/EAL6+) to resist exactly these attacks. It holds the keys and performs the signatures internally, so even full control of the microcontroller does not reveal them.

CANDIDATE	STRENGTHS	CONSIDERATIONS
Microchip ATECC608B	Cheap, I²C, P-256 ECDSA and ECDH in hardware, monotonic counters, widely used.	Small slot count: suits a root key plus derived keys, not many stored keys. Configuration is locked once.
NXP SE050	EAL6+, P-256 and many curves, larger storage, applet model.	More complex driver; NDA-free host library available.
Infineon OPTIGA Trust M	EAL6+, P-256, shielded I²C, monotonic counters.	Smaller key store than SE050.

The jotakey key puts the third of these on its own board, next to an STM32L432 microcontroller (§7.5).

7.5 THE JOTAKEY KEY: A SECURE ELEMENT

The jotakey key (`hardware/HARDWARE.md`) pairs two chips with different jobs. The **STM32L432KC**, an Arm Cortex-M4 at 80 MHz, runs the firmware: USB, CTAP, the touch sensor, the LED and the ECDSA signatures. The **Infineon OPTIGA Trust M V1**, built on Common Criteria EAL6+ certified hardware, keeps the secrets: the root, and the signature counter. The two talk over I²C.

FEATURE	ROLE IN JOTAKEY
OPTIGA data object `0xF1D0`	Holds `K_root`, 64 bytes, as a pre-shared secret. After provisioning its access conditions say: never readable, never changeable, usable only over the shielded connection.
OPTIGA `DeriveKey`	Runs the TLS 1.2 PRF (HMAC-SHA256 inside) over `K_root` and returns the result: one credential's tag or seed (§6.3). About 50 ms each.
OPTIGA monotonic counter `0xE120`	The signature counter (invariant 2). The chip only lets it go up, survives power loss by construction, and freezes it at a threshold set to its rated 600 000 updates.
OPTIGA shielded connection	Encrypts and authenticates the I²C traffic with a 64-byte platform binding secret shared between the two chips. A probe on the bus sees ciphertext; a swapped chip on either side cannot talk to the other.
OPTIGA and STM32 random generators	Both feed every random value, mixed with SHA-512, so one weak generator is not enough to weaken the key. The STM32's generator reports its own health errors (invariant 7).
STM32 readout protection	Level 2 on production units: the debug port, booting from RAM and the factory bootloader are disabled for good, and the flash cannot be read out. The setting is irreversible.

Provisioning

Provisioning happens on the device itself, from a provisioning build of the firmware that a human starts. The secrets are generated on the device and never leave it:

1. **Pair.** Generate a 64-byte platform binding secret, write it into the OPTIGA (`0xE140`) and into the STM32's flash, and lock the OPTIGA's copy.
2. **Root.** Generate `K_root` from both random generators, write it into `0xF1D0`, and lock it.
3. **Counter and epoch.** Set the counter's threshold; write a random epoch to flash.
4. **Lock.** A human sets readout protection level 2 with STM32CubeProgrammer, over the SWD test pads. From then on the firmware is frozen: Jotakey keys, like YubiKeys, are not updated in the field. A security fix means a new key. (Signed updates are a roadmap exploration, §11.)

Development boards stop before the last step and stay open to the debugger. `AGENTS.md` forbids automation from taking any of the irreversible steps: locking OPTIGA objects or setting readout protection.

What a physical attacker gets

The STM32L432 is an ordinary microcontroller. Fault-injection attacks against STM32 readout protection have been published for several families. So we assume a well-equipped thief can, in the end, run their own code on it, with the binding secret and the epoch in hand. What they get then:

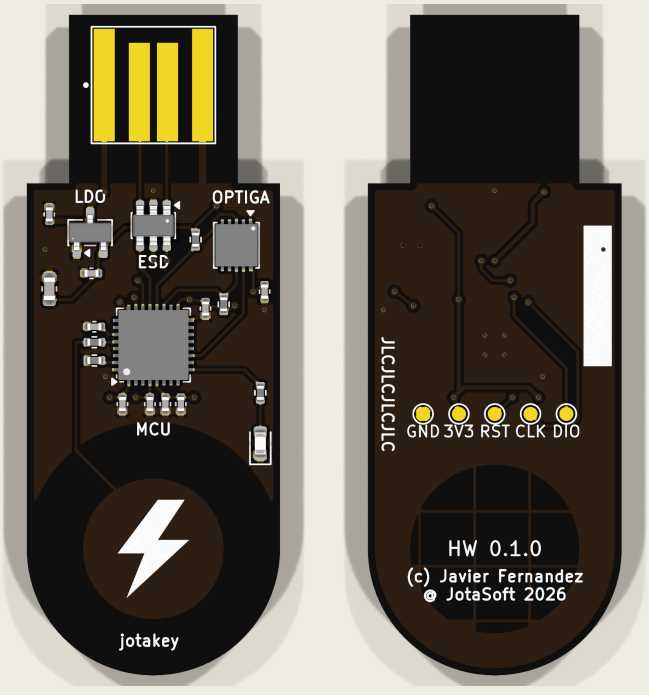
THEY CAN	THEY CANNOT
Use the key while they hold it, for any credential whose ID they present: the hardware computes its tag and seed on request.	Read `K_root`: the OPTIGA never releases it, and attacks on the OPTIGA itself are what its certification is about.
Copy a credential, if they also obtain its ID from the website (the ID is not secret).	Copy the key as a whole: no secret on the STM32 covers more than one credential, so nothing they take away lets them compute the others or mint new ones.
Make the key unusable (for example, by resetting it).	Roll the signature counter back: it lives in the OPTIGA and only goes up, so relying parties that check it can notice a copied credential.

This is the honest boundary of a two-chip design. A key whose secure element also performed the signatures would close the first row. The Trust M V1 cannot import a derived private key to sign with it, and storing one key per credential would cap the number of credentials. The threat model (§1.3) records the result as **partially defeated**.

The OPTIGA Trust M V3 adds HMAC, HKDF and AES. It was out of stock when the board's parts were chosen, so the key uses the V1, whose TLS PRF does the same job here. The two share a package and pinout, so a later board can switch; `F` would then become HMAC, as on the C6.

7.6 THE JOTAKEY KEY BOARD

The jotakey key is a two-layer board whose own edge is the USB plug. Hardware 0.1.0 (git tag `hw-v0.1.0`) is described here; `hardware/HARDWARE.md` records the part choices against their datasheets, and `hardware/MANUFACTURING.md` the order.



THE JOTAKEY KEY, HARDWARE 0.1.0, RENDERED FROM THE KICAD DESIGN: 47 × 18 MM, 2.0 MM THICK, BLACK SOLDER MASK. THE BACK IS DRAWN AS SEEN FROM BELOW.

Outline and parts

The narrow end is a USB-A plug: a 12 mm tongue carrying four gold contacts laid out as in the USB 2.0 Series "A" plug drawing, with VBUS and GND longer than the data contacts so they mate first. The board is 2.0 mm thick, so the tongue sits firmly in a receptacle, and the contacts' edge is bevelled. The 18 mm body ends in a semicircle whose centre is the centre of the touch electrode.

REF	PART	ROLE
U1	STM32L432KCU6, QFN-32	The microcontroller: USB full speed without a crystal (its clock is trimmed against the host's USB frames), the touch-sensing controller, I²C to the secure element, ECDSA signing.
U2	OPTIGA Trust M V1, USON-10	The secure element: the root secret, per-credential derivation and the signature counter (§7.5).
U3	XC6206 3.3 V LDO, SOT-23	Power from the USB 5 V.
U4	USBLC6-2SC6, SOT-23-6	ESD protection on D+, D− and VBUS, next to the contacts.
E1	10 mm copper disc	The touch electrode, user presence (invariant 1). The bolt logo is cut out of the copper.
D1	Red LED, 0603	The only display: touch requested, OK, fault.
TP1–5	Test pads, bottom	SWD (DIO, CLK, RST, 3V3, GND) for a pogo-pin jig.

Layout decisions

- **The tongue carries contacts and nothing else.** No vias and no copper on its back: it slides against the receptacle's metal shell.
- **One crossing for USB.** The plug has D− on the left; the microcontroller's pins put D− on the right. The two lines cross once, between the ESD chip and the microcontroller, where D+ drops to the back for a few millimetres. The ESD chip sits on the board's axis, and its two identical channels are assigned so the crossing stays off the tongue.
- **The touch electrode follows ST's AN4312.** Nothing on the top layer comes within 4 mm of it except its own trace (the note asks for at least 2 mm, 4–5 mm recommended), which arrives at 45°. Behind it, the back carries a ground hatch of 10 % copper instead of a solid plane, trading a little noise immunity for sensitivity. Keeping that gap is why the board is 47 mm long and not 45.5.
- **The secure element sits in the corner** next to the microcontroller's I²C pins, rotated so its clock, reset and supply pins face the microcontroller.
- **Ground pours on both layers,** tied together under each chip's exposed pad.

Silkscreen

The mask is black, which hides the contrast between copper and bare board. So the bolt cut out of the electrode's copper is also printed in white, exactly over the cut-out. The front names the chips by function (MCU, OPTIGA, LDO, ESD); the reference designators live on the assembly drawing instead, because next to 0402 parts on a board this size they cannot avoid covering pads. The back carries the hardware version, the SWD pad names, and the two places where JLCPCB prints its marks: `JLCJLCJLCJLC` for the order number and a solid block for each board's serial number.

How the board is made

The layout is reproducible from scripts in `hardware/jotakey-key/tools/`, each run with KiCad's own Python or command line. `layout.py` draws the outline and places every part; `route.py` lays down the keep-outs and the hand-routed nets (USB, power, the touch trace, the parts under the microcontroller), lets the Freerouting autorouter connect the rest, and pours the ground; `silk.py` adds the silkscreen; `fab.py` refuses to build unless the schematic and board checks pass, then writes the Gerber, BOM and placement files for JLCPCB; `render.py` draws the figures above. The hardware version is set in one place, the KiCad project variable `HW_VERSION`, which the title blocks and the back of the board display.

08 TESTING: FIVE NETS, DIFFERENT HOLES

No single technique catches everything, so Jotakey stretches several nets with different holes one above another. Chapter 9 adds the last one, proof.

NET	WHAT IT CATCHES	WHERE
Unit & scenario tests	Known behaviours, spec examples, every error path.	`crates/*/tests`, `#[cfg(test)]`
Property tests	Laws that must hold for **all** inputs: round trips, bounds, monotonicity.	`proptest` in the core
Fuzzing	Crashes, hangs and invariant violations on inputs nobody imagined.	`fuzz/` (6 targets)
Mutation testing	Gaps in the **tests themselves**: code whose breakage no test notices.	`cargo mutants`
On-device tests	What only silicon can show: the TRNG, the HMAC key, real flash, timing.	`crates/jotakey-testfw`
End-to-end	The whole chain as a relying party sees it.	`crates/jotakey-host-tests`, `scripts/fido2_check.py`

8.1 UNIT, SCENARIO AND PROPERTY TESTS

96 tests run on the host in a few seconds (`just test`). Highlights:

- **CBOR:** every integer example from RFC 8949 Appendix A; rejection of non-minimal heads, indefinite lengths, tags, floats, `undefined`, invalid UTF-8, unsorted and duplicate map keys, nesting deeper than 4; huge announced lengths fail fast instead of looping. Properties: head and integer round trips; "accepted heads re-encode to the same bytes"; "the decoder never panics on arbitrary bytes".
- **CTAP:** full register → authenticate flows with signatures verified against the public key parsed from authData; RP binding (`evil.example` gets `NO_CREDENTIALS`); tampered IDs; exclude lists; every option combination; algorithm negotiation; PIN probes; missing and wrongly typed parameters; reset window boundaries; and **fail closed**: a failing RNG, key derivation or counter write yields `CTAP1_ERR_OTHER` and **never** moves the counter.
- **Transports:** CTAPHID at every size boundary (1, 57, 58, 116, 117, 118 … 1200 bytes), channel busy and isolation, resync via INIT, sequence errors, timeouts, keepalives, cancel; BLE at every fragment size from 20 to 512. Two thousand random packet streams check that receive state stays well-formed.
- **Storage:** append and remount across sector switches; a property test that cuts power at random byte offsets, including in the middle of a sector erase, and checks that no confirmed counter value is lost or reissued.
- **Self tests:** known-answer tests for SHA-256, HMAC, HKDF and ECDSA (RFC 6979 §A.2.5), plus **fault injection**: corrupting each KAT result in turn must be detected, exactly as FIPS 140-3 expects a self-test implementation to demonstrate.

8.2 FUZZING

`cargo fuzz` (libFuzzer with AddressSanitizer) drives six targets. Each target checks security properties, not just "no crash":

TARGET	ORACLE
`cbor_decode`	No panic; anything accepted re-encodes to **exactly** the input bytes (canonical uniqueness, the theorem of §9.5, checked on real code).
`ctap_dispatch`	Raw and template-mutated requests through the authenticator. The counter moves **only** on a successful makeCredential/getAssertion completed with a confirmed touch, and by exactly one; success bodies are canonical CBOR; nothing stays parked.
`ctaphid_stack`	Arbitrary 64-byte packet streams on several channels, with clock jumps and absent users, through CTAPHID **and** the authenticator. Buffers stay within bounds, no request is left unanswered, output is bounded.
`ble_stream`	Arbitrary GATT write sequences and fragment sizes, through the BLE framer and the authenticator.

TARGET	ORACLE
`serial_deframe`	Arbitrary byte streams through the COBS/CRC deframer; every report emitted is a valid frame.
`credential_unwrap`	Accepted IDs are exactly what `wrap` mints; minted IDs always unwrap.

Smoke runs during development covered roughly 8 million CBOR, 6 million CTAPHID, 4 million BLE, 4.7 million serial, 23 million credential and 2 million full CTAP executions, with no failures. Corpora grow over time. Inputs captured from real browsers through the bridge make good seeds.

8.3 MUTATION TESTING

A test suite can pass while testing very little. **Mutation testing** measures this. `cargo mutants` makes hundreds of small deliberate bugs in the code: it flips `<` to `<=`, deletes a `!`, replaces a function body with a constant, deletes a `match` arm. It then runs the test suite against each one. A mutant that survives is a change to the code that no test notices.

RUN	MISSED	WHAT CHANGED
First run	61 / 459	Revealed untested boundaries (user-ID length 64/65, reset window at exactly 10 000 ms), optional members whose **type** was never checked, CTAPHID resync corners, status codes only compared with themselves, and self tests whose failure branch could never run.
After fixes	0	New targeted tests; an unreachable branch replaced by a compile-time assertion; dead API removed; fault injection added to the self tests.
Storage added	11	The new record log passed its functional tests, yet its on-flash **format**, CRC rejection and sector-switch schedule were unpinned: mutants that wasted slots or erased too often survived. Fixed with a golden layout test, an erase-count schedule test and corruption tests.
Current	**0 / 530**	452 caught, 72 unviable (do not compile), 6 time out (infinite loops, counted as caught).

The policy (`AGENTS.md` §4) is **zero unexplained survivors**. The only excluded class is documented in `.cargo/mutants.toml`: replacing `|` with `^` where the operands are disjoint bit fields (`cmd | 0x80` with `cmd < 0x80`), which produces identical values; one comparison between sequence numbers that are unique by construction; and the mock flash, which is test scaffolding. Mutants that make code loop forever are caught by timeouts.

> **WHY BOTHER, WITH PROOFS AVAILABLE?**
> Mutation testing found that the self tests' failure paths were never exercised: a broken `run_kats` that always returned `Ok` would have passed every test. Proofs about the protocol model would not have noticed either, because the self tests are not part of it. Each net has holes the others cover.

8.4 ON-DEVICE TEST FIRMWARE

Some properties exist only on silicon. `crates/jotakey-testfw` is a separate firmware built with `embedded-test` and run by `probe-rs` through the C6's built-in USB-JTAG. No extra debug probe is needed. Each test is flashed, the chip is reset, the test runs, and pass/fail comes back over semihosting with logs over RTT.

TEST	CHECKS
`known_answer_tests_pass_on_target`	The KATs on the RISC-V target (endianness, compiler, crypto backends).
`injected_faults_are_detected_on_target`	Each injected self-test fault is detected on the target.

TEST	CHECKS
`trng_health_and_statistics`	SP 800-90B start-up test; 64 KiB through the continuous tests; byte χ^2 and monobit within ±6σ.
`efuse_root_key_hmac`	HMAC peripheral with eFuse KEY0: deterministic, label-separated. **Skipped with a warning** on unprovisioned boards.
`record_log_on_real_flash`	600 appends across sector switches on real NOR flash; a torn record is ignored; uses a scratch area that never touches production data.
`ctap_register_and_authenticate_on_target`	Full makeCredential + getAssertion on the chip with the real TRNG; the signature verifies; no touch, no signature.
`ecdsa_timing_budget`	Sign + verify under 400 ms per operation at full clock.
`status_led_accepts_frames`	WS2812 frames via RMT.
`user_presence_button` (interactive)	Press BOOT within 30 s (`--features interactive`).

8.5 END TO END

`jotakey-host-tests` contains a CTAPHID **client** written independently of the device code. It has its own CBOR codec, so a bug that is symmetric in the device's encoder and decoder cannot hide. It also contains relying-party checks as a WebAuthn server would perform them (authData layout, COSE key, packed self-attestation, signatures, counters). The client runs against two devices:

- **the simulator**, an in-process device wired exactly like the firmware's USB task: serial framing → CTAPHID → authenticator, with keepalives while "waiting for the touch". This runs in every `just test`.
- **a real Jotakey** through `jotakey-bridge` and `/dev/hidrawN` (ignored by default; run with `just host-tests` while touching the button when asked).

Finally, `scripts/fido2_check.py` drives the device with Yubico's **python-fido2**, both as client **and** as relying-party server. That is a completely independent implementation of both ends of the protocol.

8.6 STATIC CHECKS

- `#![forbid(unsafe_code)]` in the core; Clippy denies `unwrap`, `expect`, `panic!`, unchecked indexing and unchecked arithmetic in shipped core code.
- `overflow-checks = true` in every profile, release included.
- `cargo deny`: no known-vulnerable or yanked crates, allowed licences only, crates.io only. Exceptions are documented with reasons: two unmaintained crates that run only at build time (a proc-macro and a build-script YAML parser) and never ship in the firmware.

09 FORMAL VERIFICATION, FROM THE GROUND UP

This chapter assumes no background in formal methods. It explains what formal verification is, surveys the techniques, teaches enough Lean 4 to read Jotakey's proofs, walks through the main ones, and is explicit about what they do **not** guarantee. Chapter 10 then goes through every theorem in detail.

9.1 WHAT A PROOF BUYS YOU

Every test in chapter 8 checks **some** inputs. A fuzzer may run a billion of them, but a CTAP message has more possible values than there are atoms in the universe. As Dijkstra put it in 1970, "program testing can be used to show the presence of bugs, but never to show their absence."

A **formal proof** is different in kind. It is a mathematical argument, checked mechanically by a computer, that a property holds for **every** input, **every** sequence of events, **every** possible state. When we prove "no execution of the authenticator state machine produces a signature without a confirmed touch", we are not sampling executions. We have shown that such an execution cannot exist.

The catch is the word **model**. A proof is about a precise mathematical description of a system. If that description differs from the real system, the proof says nothing about the difference. Much of the craft lies in choosing what to model, and in closing the gap between model and code (§9.7).

> **TESTING SAMPLES BEHAVIOURS. PROOF COVERS ALL OF THEM, FOR THE MODEL IT IS ABOUT.**

9.2 THE LANDSCAPE OF TECHNIQUES

Formal methods form a spectrum, from cheap and automatic to expensive and complete. They are complements, not rivals. Jotakey uses several at once.

TECHNIQUE	WHAT IT DOES	EXAMPLES
Type systems	Rule out whole classes of errors at compile time. Rust's ownership types rule out use-after-free and data races in safe code.	Rust, Haskell, OCaml
Property-based testing	Check a stated property on many random inputs, shrinking failures to a minimal example.	proptest, QuickCheck, Hypothesis
Fuzzing	Coverage-guided random inputs search for crashes and violated assertions.	libFuzzer, AFL++, cargo-fuzz
Model checking	Exhaustively explore every state of a finite (or bounded) model, and produce a counterexample trace if a property fails.	TLA+/TLC, SPIN, CBMC, **Kani** (Rust)
Abstract interpretation	Over-approximate all behaviours to prove the absence of run-time errors.	Astrée (Airbus flight software), MIRAI
Deductive verification	Annotate code with pre/postconditions and invariants; an SMT solver discharges the proof obligations automatically.	Dafny, Why3, Frama-C, **Verus**, **Creusot**, **Prusti**
Interactive theorem proving	Write definitions and proofs in a rich logic; a small kernel checks every step. Most expressive and most laborious.	Rocq (Coq), Isabelle/HOL, **Lean 4**, F*

Landmark results show what the strong end can reach:

- **seL4** (Isabelle/HOL, 2009–): an operating-system microkernel whose C implementation is proved to refine its specification, down to the compiled binary.
- **CompCert** (Coq): a C compiler proved never to miscompile. In a large compiler-testing study, it was the only compiler in which no wrong-code bugs were found in the verified parts.
- **HACL*** and **Fiat-Crypto**: verified cryptographic code now running in Firefox, the Linux kernel, Chrome and BoringSSL.
- **Amazon Web Services**: TLA+ for distributed protocols; the Cedar authorization language, whose specification and proofs are written in Lean.

9.3 LEAN 4 IN TWENTY MINUTES

Lean 4 is both a functional programming language and a proof assistant. That duality is exactly what we need: the **same** definitions can be run (as an executable specification) and reasoned about (as mathematics).

Data and functions

Types are defined by listing their constructors, and functions by pattern matching:

DATA AND FUNCTIONS

```
inductive PresenceResult where       -- result of waiting for a touch
  | confirmed | timeout | cancelled

def toLittleEndian : Nat → Nat → List Nat   -- the low `width` bytes, least significant first
  | 0,          _     => []
  | width + 1, value => value % 256 :: toLittleEndian width (value / 256)

#eval toLittleEndian 2 1000          -- [232, 3]
```

Lean insists that every function **terminates** and handles **every** case. A function that could loop or crash is rejected at definition time. For our CBOR decoder that is already a theorem: the definition compiles, so the decoder terminates on every input.

Propositions are types, proofs are programs

The central idea, the **Curry–Howard correspondence**, is that a statement is a type, and a proof of the statement is a value of that type. `2 + 2 = 4` is a type. `rfl` ("by reflexivity") is a value of it. A theorem is a definition whose type is the statement:

A FIRST THEOREM

```
theorem toLittleEndian_length (width value : Nat) :
    (toLittleEndian width value).length = width := by
  induction width generalizing value with
  | zero          => rfl                        -- the empty list has length 0
  | succ width ih => simp [toLittleEndian, ih]   -- one byte, plus `width` by induction
```

The part after `by` is a **tactic script**. Instead of writing the proof term by hand, we issue commands that build it step by step:

TACTIC	WHAT IT DOES
`intro`	Assume the premise of an implication ("suppose P …").
`cases` , `match`	Split into one case per constructor ("either it timed out, or …").
`induction`	Prove for 0 (or `[]`), then for `n+1` assuming `n` (or `x :: xs` assuming `xs`).
`simp`	Rewrite with known equations until the goal simplifies away.
`omega`	Decide linear arithmetic over integers and naturals, including `/` and `%` by constants.
`rfl` , `decide`	Close a goal that holds by computation.
`exact` , `apply`	Use an existing lemma to finish, or reduce, the goal.

Who checks the checker?

Tactics are elaborate and could contain bugs. That does not matter. Every tactic only **proposes** a proof term, and a small, separate **kernel** type-checks the final term. Only the kernel must be trusted (the "de Bruijn criterion"). Lean also records which **axioms** a proof relies on. `#print axioms` lists them, and `formal/AxiomAudit.lean` checks every Jotakey theorem:

LAKE ENV LEAN AXIOMAUDIT.LEAN (EXCERPT)

```
'Jotakey.Authenticator.signature_requires_confirmed_touch' depends on axioms:
    [propext, Quot.sound]
'Jotakey.Authenticator.trace_signature_counters_increase' depends on axioms:
    [propext, Classical.choice, Quot.sound]
'Jotakey.CtapHid.fragmented_message_is_reassembled' depends on axioms: [propext, Quot.sound]
'Jotakey.Cbor.decodeHead_accepts_only_canonical' depends on axioms: [propext, Quot.sound]
```

These are the three standard axioms of Lean's logic (propositional extensionality, quotient soundness, and choice), the same ones all of mathlib uses. Two markers would mean trouble: `sorryAx` , which records an

unfinished proof, and `Lean.ofReduceBool`, which records a proof that trusts the compiler. Neither appears, and `just lean` fails the build if either ever does.

9.4 MODELLING: DECIDING WHAT TO PROVE ABOUT

A model is a deliberate simplification. Jotakey's models follow three rules:

1. **Mirror the code's structure.** The Lean `receivePacket` function for CTAPHID has the same cases, in the same order, as `CtapHid::handle_packet`. Reviewers can compare them line by line.
2. **Abstract what the property does not depend on.** The authenticator model does not parse CBOR or compute ECDSA. A request arrives already classified as `makeCredential relyingParty`, `getAssertion relyingParty credential`, `rejected` and so on. "No signature without a touch" does not depend on how parsing works, so the parser is not part of that model. (It has its own model, below.)
3. **Make assumptions explicit.** Cryptography cannot be proved in Lean. The credential model takes the MAC as a **parameter**, and any theorem that needs a cryptographic property states it as a hypothesis, visible in the theorem's signature.

9.5 CASE STUDY: CBOR HAS EXACTLY ONE ENCODING

The first model, `formal/Jotakey/Cbor/`, mirrors the decoder of §5.1. The key theorem says that whatever bytes the decoder accepts are **exactly** the canonical encoding of what it returned:

FORMAL/JOTAKEY/CBOR/HEADS.LEAN

```
theorem decodeHead_accepts_only_canonical (bytes : List Nat) (majorType argument : Nat)
    (rest : List Nat)
    (hBytes : ∀ byte ∈ bytes, byte < 256)                    -- these really are bytes
    (h : decodeHead bytes = some (majorType, argument, rest)) : -- the decoder accepted them
    bytes = encodeHead majorType argument ++ rest             -- … so they ARE canonical
```

Read it as: "for every byte list `bytes`, if the decoder accepts it, then it is exactly the canonical encoding of the result, followed by the unread rest." Two different byte strings can therefore never decode to the same head. The ambiguity attacks of §5.1 are impossible for any parser that implements this model.

The proof splits on the width of the argument (immediate, 1, 2, 4 or 8 bytes). In each case it uses a lemma about big-endian numbers, `bigEndian_bytes_roundtrip`: reading `k` bytes as a number and writing the number back as `k` bytes gives the same bytes. The minimality check is what makes the case split work. A value accepted from the 2-byte form is ≥ 256, so `encodeHead` also chooses the 2-byte form. Without that check in the decoder, the theorem would be **false**, and Lean would refuse the proof. That is the point: the proof would not have gone through for a lax decoder.

Its companion, `decodeHead_inverts_encodeHead`, proves the round trip in the other direction. The full validator `skipItem`, which handles maps, key ordering, UTF-8 and depth, is defined in `Cbor.lean`, next to it. Lean's termination checker accepts it, so the decoder provably terminates on every input.

9.6 CASE STUDY: THE SECURITY STATE MACHINES

No signature without a touch, ever

`formal/Jotakey/Authenticator.lean` models `Authenticator::process` and `complete` as pure functions `processRequest` and `completePresence` on a `State` (pending command, signature counter, reset epoch, fault flag), driven by `Event`s: requests and presence results. A **trace** is any list of events, in any order, of any length. The theorems quantify over all of them:

FORMAL/JOTAKEY/AUTHENTICATOR.LEAN

```
-- Invariant 1.
theorem signature_requires_confirmed_touch (state : State) (event : Event)
    (relyingParty counter : Nat)
    (hSigned : (handleEvent state event).2 = .signature relyingParty counter) :
    event = .presence .confirmed ∧ state.faulted = false ∧
      (state.pending = .makeCredential relyingParty ∨
        ∃ credential, state.pending = .getAssertion relyingParty credential)

theorem one_touch_signs_at_most_once (state : State) (first second : PresenceResult)
    (relyingParty counter : Nat) :
    (completePresence (completePresence state first).1 second).2 ≠
      .signature relyingParty counter

-- Invariant 2, over every possible trace.
theorem trace_signature_counters_increase (state : State) (events : List Event) :
    (signatureCounters (runTrace state events).2).Pairwise (· < ·)
```

In words:

- A signature comes **only** from a confirmed presence event that completes a parked makeCredential or getAssertion for **that** relying party, on a healthy device.
- One touch completes at most one command. A second “confirmed” event right after produces nothing.
- In **any** interleaving of requests, touches, timeouts, cancellations and resets, the counters of emitted signatures form a strictly increasing sequence.

`epoch_change_requires_reset_in_window` proves invariant 6 over traces. If the epoch ever changes, the trace contains a reset request that arrived within 10 seconds of power-up. `faulted_stays_faulted_and_never_signs` proves that the fault state of invariant 7 is absorbing.

Channel isolation and exact reassembly

`formal/Jotakey/CtapHid.lean` models the CTAPHID receive path. `receivePackets_ignores_other_channels` proves that any sequence of packets from other channels leaves the owning channel’s transaction exactly as it was. `receivePacket_preserves_well_formed` proves that buffers never exceed the announced length or the 1200-byte limit. `fragmented_message_is_reassembled` proves that a message fragmented the way a host must fragment it is delivered intact:

FORMAL/JOTAKEY/CTAPHID.LEAN

```
theorem fragmented_message_is_reassembled (channel command : Nat) (message : List Nat)
    (hMaxSize : message.length ≤ maxMessageSize) :
    receivePackets .idle (fragmentMessage channel command message) = .busy channel message
```

While writing this proof, Lean pointed out that a hypothesis we had added, `0 < message.length`, was never used. The theorem also holds for empty messages. Proof assistants often teach you something about your own design this way.

Credential binding, with the assumption on the table

FORMAL/JOTAKEY/CREDENTIAL.LEAN

```
theorem unwrap_accepts_only_wrapped_ids (key relyingParty credentialId nonce : List Nat)
    (h : unwrap mac key relyingParty credentialId = some nonce) :
    credentialId = wrap mac key relyingParty nonce ∧ nonce.length = nonceLength

theorem credential_bound_to_relying_party
    (hNoCrossSiteCollision : ∀ key party party' nonce,          -- the cryptographic
      party ≠ party' → mac key party nonce ≠ mac key party' nonce)  -- assumption
    (key relyingParty otherParty nonce : List Nat) (hNonceLength : nonce.length = nonceLength)
    (hDifferentParty : relyingParty ≠ otherParty) :
    unwrap mac key otherParty (wrap mac key relyingParty nonce) = none
```

`unwrap_accepts_only_wrapped_ids` needs no assumption at all: the construction accepts nothing but its own outputs. `credential_bound_to_relying_party` needs the MAC not to collide across relying parties, and

says so in its signature. That division is what a proof should give you: everything that can be established by logic is, and the rest is named.

9.7 CLOSING THE GAP BETWEEN MODEL AND CODE

The proofs are about Lean functions. The device runs Rust. How do we know they agree? There are three increasingly strong answers. Jotakey implements the first two today and plans the third.

1. Faithful transcription and review
Each model mirrors its Rust counterpart's structure (§9.4), and every Rust module states which Lean file models it. Changes to one require changes to the other (`AGENTS.md` §4). This is the weakest link: it relies on human care.

2. Differential testing against an executable specification
Lean definitions are executable. The very functions the CBOR theorems are about are compiled into a program, and a Rust test asks it and the Rust decoder the same 101 536 questions. Every answer must match. Any future change to the Rust decoder that departs from the proved model fails this test. Differential testing does not prove equivalence, but it ties the proofs to the code with a mechanical check rather than review alone. §9.8 shows how it works.

The fuzz targets play a similar role for the other models: their oracles are the theorem statements ("counter moves only on a confirmed signature", "reassembly never exceeds the announced length") checked on the real Rust code.

3. Translating the Rust itself: Aeneas (planned experiment)
Aeneas (Ho & Protzenko, Inria/Microsoft) translates Rust into pure Lean functions, using the **Charon** front end to extract the compiler's internal representation. Rust's ownership discipline is what makes this possible: a program without aliasing of mutable data can be expressed as a pure function that takes a value and returns the updated one. With Aeneas, we would no longer **write** `receivePacket` in Lean. We would **generate** it from `ctaphid.rs` and prove that the generated function refines the hand-written model (or prove the theorems directly about it). That removes link 1 entirely. Microsoft has used this route to verify parts of its Rust cryptography library (SymCrypt).

Aeneas supports a subset of Rust. `jotakey-core` is written to stay close to it: no trait objects, no interior mutability, simple loops with explicit bounds (`AGENTS.md` §4). A few closures remain in the CTAP layer (`find_in_list`, `write_success`). Rewriting them as plain loops is the first step of the experiment. The experiment is recorded in the roadmap. Other tools could close the same gap differently:

TOOL	APPROACH
Aeneas	Rust → pure Lean (or F*, Coq, HOL4); interactive proofs about the translation.
hax	Rust → F* or Rocq; used for the verified ML-KEM in the `libcrux` library.
Verus	Annotated Rust, verified by an SMT solver; good for low-level code with loop invariants.
Kani	Bounded model checking of Rust (CBMC); proves absence of panics and assertion failures for bounded inputs, with no annotations.
Creusot, Prusti	Deductive verification of annotated Rust (Why3 and Viper backends).

Kani is the cheapest next step. It could prove, for all inputs up to a bound, that `Decoder::skip` and `CtapHid::handle_packet` never panic, directly on the Rust code.

9.8 DIFFERENTIAL TESTING, STEP BY STEP

Differential testing gives the same input to two implementations of the same thing and compares their answers. It needs no expected outputs: each implementation is the other's oracle, and any disagreement means at least one of them is wrong. The technique is old (McKeeman named it in 1998), and it is how tools like Csmith found hundreds of bugs in GCC and LLVM by comparing compilers against each other.

Jotakey uses a stronger version. One side is the shipped Rust decoder. The other side is not a second hand-written decoder but **the Lean model itself**, the functions `decodeHead` and `validate` that §9.5 proves theorems about. A test failure therefore means one precise thing: the Rust code does something the proved model does not.

Step 1: the model becomes a program

Lean is a programming language as well as a prover, so the same definitions that appear in the theorems can be compiled. `formal/Main.lean` wraps them in a short program, `jotakey-spec`, that reads one question per line and answers it:

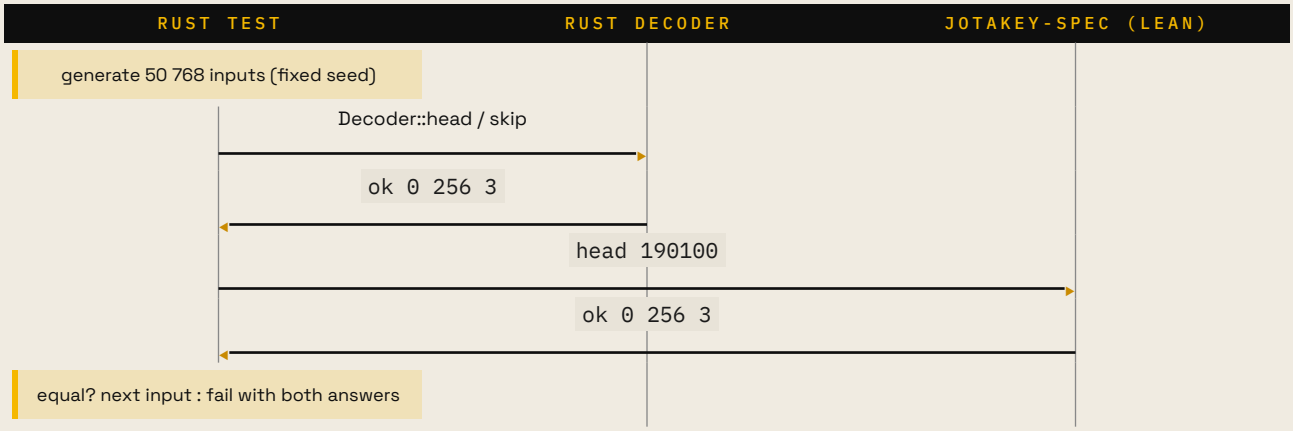
A CONVERSATION WITH JOTAKEY-SPEC

```
head 1817            →  err          0x18 says "1-byte argument", but 0x17 = 23 fits in the
                                      initial byte: non-minimal, rejected
head 1818            →  ok 0 24 2    major 0 (unsigned), value 24, 2 bytes consumed
head 190100          →  ok 0 256 3   2-byte argument 0x0100 = 256
validate a201000200  →  ok 5         map {1: 0, 2: 0}: keys in canonical order
validate a202000100  →  err          map {2: 0, 1: 0}: keys out of order
validate 5f          →  err          indefinite-length byte string: not canonical
```

`head` asks for one CBOR head: major type, argument, and how many bytes it used. `validate` asks whether the input starts with one complete, canonical data item, and how long it is. The program contains no decoding logic of its own. Hex parsing and printing are the only code between the question and the proved definitions.

Step 2: the same questions to Rust

`crates/jotakey-core/tests/differential.rs` asks `Decoder::head` and `Decoder::skip` the same questions and prints the answers in the same format. It starts `jotakey-spec` as a child process, writes every question to its input, reads every answer back, and compares the two strings exactly.



ONE ROUND OF THE DIFFERENTIAL TEST. ALL QUESTIONS ARE SENT IN ONE BATCH.

Agreement means more than “both accepted”. They must agree on the decoded values and on the **number of bytes consumed**. A decoder that accepted the right item but stopped one byte early would leave that byte to be misread as the next item. That is how parser differentials are exploited, so it is checked.

Step 3: questions worth asking

Purely random bytes are almost always rejected in the first byte or two, so they would test little more than the error path. The test builds four families of inputs:

FAMILY	INPUTS	WHAT IT EXERCISES
Generated values	20 000	Random nested arrays, maps, integers, byte and text strings, booleans and null, encoded by the Rust encoder. Half of the integers are boundary values: 0, 23, 24, 255, 256, 65 535, 65 536, $2^{32}-1$, 2^{32}, $2^{63}-1$, 2^{63}, $2^{64}-2$, $2^{64}-1$. These exercise the accepting paths deep inside maps and arrays.
Mutated values	20 000	Each generated value again, with up to two random edits: overwrite a byte, truncate, append, swap two neighbours, or insert a byte known to be dangerous (`5f` indefinite length, `ff` break, `f7` undefined, `c0` tag, `f9` half float, `1c` reserved, `18` one-byte head). These land just beside valid inputs, where bugs live.
Random bytes	10 000	0 to 11 random bytes.

FAMILY	INPUTS	WHAT IT EXERCISES
All heads	768	Every possible initial byte 0x00–0xFF, alone, followed by eight `00` bytes, and followed by eight `ff` bytes. Together these cover every head shape, every width and both extremes of every argument.

Each input is asked both ways, as `head` and as `validate`, which gives 101 536 questions. They take about a quarter of a second. The generator uses a fixed seed, so a failure is reproducible. The test also checks that more than a tenth of the answers are acceptances; at present 80 069 are. Otherwise a broken generator could produce only garbage, both sides would answer `err` to everything, and the test would pass while testing nothing.

Seeing it catch a bug

To check that the test has teeth, we broke the Rust decoder on purpose. In `Decoder::head`, a 2-byte argument must be at least 256, otherwise it should have used one byte. We changed `if v <= 0xFF` to `if v < 0xFF`, a one-character off-by-one that lets exactly one non-canonical encoding through. The differential test failed at once:

CARGO TEST -P JOTAKEY-CORE --TEST DIFFERENTIAL

```
assertion `left == right` failed: Rust and Lean disagree on `validate 3900ff4bc5d540`
  left: "ok 3"
 right: "err"
```

`39` is major type 1 (negative integer) with a 2-byte argument, `00 ff`. That is 255, which fits in one byte, so the encoding is not canonical. The broken Rust accepted it (`ok 3`); the model refused it (`err`). The input came from the “mutated values” family.

The same bug was also caught by a unit test, `rejects_non_minimal_heads`, which checks this exact boundary (`19 00 ff`) by hand. The random property test `accepted_heads_are_canonical` did **not** catch it: random bytes rarely land on a value of exactly 255 after a `39`. Each net has different holes.

What a pass does and does not mean

A pass means: on 101 536 inputs, chosen to sit on the boundaries, the shipped decoder gives exactly the answers of a function proved to have canonical uniqueness, termination and the round-trip property. It is the strongest link between the proofs and the Rust code that Jotakey has today. It does not mean:

- **Equivalence on all inputs.** The test samples. An input outside the four families could still differ. The Aeneas route of §9.7 would close this gap with a proof.
- **Coverage of the other models.** Only the CBOR decoder has an executable specification so far. The state-machine models are linked to the Rust code by fuzz oracles that check the theorem statements (chapter 8), not by differential tests.
- **Protection against a shared misreading of the standard.** If the model and the Rust code both misunderstand RFC 8949 in the same way, they agree and the test passes. The theorems help here because they state the intended properties independently of either implementation. Reviewing the model against the standard remains a human job.

The Lean compiler and runtime that build `jotakey-spec` are also trusted, but they would have to fail in a way that exactly mimics a Rust bug for the test to wrongly pass.

Running it takes two commands. Without the spec binary the test reports that it skipped; setting `JOTAKEY_REQUIRE_SPEC=1` turns a missing binary into a failure, which is the right setting for continuous integration.

RUNNING THE DIFFERENTIAL TEST

```
cd formal && lake build jotakey-spec && cd ..
cargo test -p jotakey-core --test differential -- --nocapture
# differential: 101536 queries agree (80069 accepted)
```

9.9 WHAT REMAINS TRUSTED

A verified system is only as trustworthy as its **trusted computing base**, the parts that are assumed rather than proved. For Jotakey:

TRUSTED COMPONENT	WHY WE ACCEPT IT, AND MITIGATIONS
Lean kernel + 3 axioms	Small, widely used, independently re-checkable (external checkers exist).
The models' faithfulness	Mirrored structure, review, the differential test, fuzz oracles. Aeneas would remove most of this.
The specification itself	A proof that the code meets a wrong spec is worthless. Our specs are short, written in the vocabulary of the CTAP text, and reviewed against it.
rustc + LLVM	Not verified. Overflow checks stay on in release builds. On-device tests run the actual binary.
RustCrypto `p256`, `sha2`, `hmac`, `hkdf`	Audited, widely deployed; checked at boot against published test vectors.
Cryptographic hardness	ECDLP on P-256, SHA-256 collision resistance. Named explicitly (e.g. `credential_bound_to_relying_party`'s hypothesis).
The silicon	See chapter 7: fault injection is outside what any proof here can address.

9.10 WORKING WITH THE PROOFS

EVERYDAY COMMANDS

```
cd formal && lake build                    # check every proof (seconds)
lake env lean AxiomAudit.lean              # which axioms each theorem uses
lake exe jotakey-spec <<< "validate a10102" # run the executable specification
just lean                                  # build + reject sorry, axioms, native_decide
just diff                                  # Rust vs Lean differential test
```

To change a model, edit the Lean file, run `lake build` and read the first error. Lean shows the exact goal it could not prove, which is usually the precise case your change broke. To **add** a property, state it as a `theorem`. If you get stuck, write `sorry` to continue exploring. Lean itself only warns about `sorry`, but `just lean` **rejects** it, so an unfinished proof cannot pass the project's checks.

10 THE PROOFS, ONE BY ONE

Chapter 9 explained what formal verification is and showed the headline results. This chapter is the companion for reading the proofs themselves. It goes through every theorem in `formal/`, in the order the files are best read, and for each one gives the statement, what it means, how the proof works, and which later theorem relies on it. The source files carry the same explanations in their doc comments (**Proves** and **How**), so the chapter and the code can be read side by side.

There are 45 theorems in four files. Most are **lemmas**: stepping stones, proved because a bigger proof needs them. Following the dependencies from each lemma to the theorems that use it, we eventually reach theorems that nothing else uses. These are the **key theorems**: the conclusions the whole development exists to establish. There are twelve.

KEY THEOREM: WHAT IT SAYS	WHY IT MATTERS
`decodeHead_accepts_only_canonical` · invariant 4 Whatever the decoder accepts is exactly the canonical encoding of what it returned.	No value has two accepted encodings, so two parties can never read the same bytes differently. This removes a whole class of parser-confusion attacks.

KEY THEOREM: WHAT IT SAYS	WHY IT MATTERS
`decodeHead_inverts_encodeHead` · invariant 4 Decoding an encoded head returns what was encoded and leaves the rest untouched.	The strictness above does not reject anything legitimate: every canonical head is accepted.
`decodeHead_consumes_input` · invariant 4 Every accepted head consumes at least one byte.	Reading cannot stall in place. This is the basis of the decoder's time bound.
`receivePackets_ignores_other_channels` · invariant 5 Packets on other channels never change a transaction in progress.	Malware talking to the key on its own channel cannot inject into, truncate or abort another program's request.
`receivePackets_preserves_well_formed` · invariant 4 After any packet sequence, buffers are within the announced length and 1200 bytes.	Memory use is bounded by a constant, whatever the host sends. Firmware without a heap needs exactly this.
`fragmented_message_is_reassembled` · functional A correctly fragmented message is delivered intact.	It rules out a model that is "safe" only because it accepts nothing (see below).
`one_touch_signs_at_most_once` · invariant 1 One touch completes at most one command.	Malware cannot queue a second signature behind a touch the user gave for something else.
`trace_signature_counters_increase` · invariant 2 In any sequence of events, signature counters strictly increase.	Relying parties use the counter to detect cloned keys. A repeated or decreasing counter would raise false alarms or hide a clone.
`epoch_change_requires_reset_in_window` · invariant 6 The reset epoch changes only if a reset request arrived in the first 10 s after power-up.	Software on the host cannot wipe every credential remotely. A reset needs the key to be replugged.
`faulted_stays_faulted_and_never_signs` · invariant 7 A faulted device stays faulted and never signs.	Fail closed: after a self-test failure, no signature is produced from possibly broken crypto.
`unwrap_recovers_nonce` · functional A credential ID unwraps, at its own site, to the nonce it was minted with.	Legitimate credentials keep working. This is the anti-vacuity partner of the next theorem.
`credential_bound_to_relying_party` · invariant 3 An ID minted for one relying party is rejected at any other, assuming the MAC does not collide.	Phishing resistance at the credential level: an ID is useless outside the site that created it.

The invariant numbers are those of `AGENTS.md`, §2. Two of the key theorems prove no security invariant: `fragmented_message_is_reassembled` and `unwrap_recovers_nonce`. They are there because safety properties are easy to satisfy by doing nothing. A device that dropped every packet would trivially satisfy channel isolation and bounded memory. A device that rejected every credential ID could never sign for the wrong site. **Functional** theorems like these two show the model also does its job, so the safety theorems are not true for an empty reason.

"Nothing else uses it" is a good way to find the conclusions, but it is not a definition of importance. Four lemmas deserve the same attention, even though another theorem builds on them:

- `signature_requires_confirmed_touch` **is** invariant 1, the most important property of the device: no signature without a confirmed touch, for the parked relying party. It drops off the key list only because `one_touch_signs_at_most_once` is proved from it.
- `receivePacket_preserves_well_formed` is the statement for a single packet. It matches what each call of the Rust `handle_packet` must preserve, and `receivePackets_preserves_well_formed` extends it to any sequence of packets.
- `epoch_change_requires_confirmed_touch` is the other half of invariant 6: a reset needs a **touch**, as well as the time window.
- `unwrap_accepts_only_wrapped_ids` says that `unwrap` accepts nothing but `wrap`'s own outputs, with no assumption about the MAC at all. `credential_bound_to_relying_party` is a corollary.

All twelve key theorems and these four are listed in `formal/AxiomAudit.lean`, which checks on every build that they rest only on Lean's three standard axioms (§9.3). The two panels below show which theorem is proved from which; key theorems are at the left margin.

```
WHO USES WHOM: CBOR HEADS (EACH LINE IS PROVED USING THE LINES BELOW IT)

decodeHead_inverts_encodeHead
└ readArgument_reads_encoded
   ├ toBigEndian_length — toLittleEndian_length
   └ bigEndian_number_roundtrip — littleEndian_number_roundtrip
decodeHead_accepts_only_canonical
├ reencoded_prefix_restores_bytes
│  └ bigEndian_bytes_roundtrip — littleEndian_bytes_roundtrip
├ prefix_value_fits_width
│  └ fromBigEndian_fits_width — fromLittleEndian_fits_width
├ encodeHead_one_byte_form, _two_byte_form, _four_byte_form, _eight_byte_form
└ decodeHead_extended_facts — readArgument_accepted_facts
decodeHead_consumes_input
└ decodeHead_extended_facts
```

```
WHO USES WHOM: CTAPHID, AUTHENTICATOR, CREDENTIALS

receivePackets_ignores_other_channels — receivePacket_ignores_other_channels
receivePackets_preserves_well_formed
└ receivePacket_preserves_well_formed — deliverIfComplete_is_well_formed
fragmented_message_is_reassembled
├ continuations_complete_message (and itself, on a shorter remainder)
└ trimming_keeps_host_payload, continuationPackets_of_empty

one_touch_signs_at_most_once
├ signature_requires_confirmed_touch — processRequest_never_signs_with_presence
├ completePresence_signs_with_presence
└ completePresence_clears_pending
trace_signature_counters_increase
├ trace_signature_counters_bounded
│  └ runTrace_never_lowers_counter — handleEvent_never_lowers_counter
└ signature_carries_next_counter
epoch_change_requires_reset_in_window
└ epoch_change_requires_confirmed_touch, parked_reset_was_requested_in_window,
   completePresence_clears_pending
faulted_stays_faulted_and_never_signs   (direct)

unwrap_recovers_nonce                   (direct)
credential_bound_to_relying_party — unwrap_accepts_only_wrapped_ids
```

The reading order follows the files: numbers and bytes first (§10.2), then CBOR heads (§10.3) and the validator built on them (§10.4), CTAPHID (§10.5), the authenticator state machine (§10.6) and credential IDs (§10.7). §10.8 lists what the proofs leave out, and §10.9 maps every Rust name to its Lean counterpart. The Lean syntax is introduced in the same order in the comments of the source files, and `formal/Jotakey.lean` repeats the order.

10.1 BEFORE YOU START

The best way to read a Lean proof is in an editor. With the Lean 4 extension for VS Code, open the `formal/` folder, place the cursor on any line of a proof, and the **infoview** panel shows the hypotheses available at that point and the goal still to be proved. A proof that seems cryptic on paper becomes a sequence of small, visible steps. The tactics used are summarised in §9.3; `omega` (linear arithmetic) and `simp` (rewriting) do most of the routine work.

Four conventions run through all the files:

- **Names say what things are.** Functions are named after what they do (`receivePacket`, `completePresence`), theorems after what they state (`decodeHead_accepts_only_canonical`), and variables after what they hold (`width`, `relyingParty`, `nextSequence`). The one deliberate exception is `ih`, the conventional name for an induction hypothesis. The Rust code follows the same rule in its own conventions; §10.9 translates between the two.
- **Hypotheses are arguments.** `(hFits : value < 256 ^ width)` in a theorem's signature means "assuming the value fits in `width` bytes". Hypothesis names start with `h` and say what they assume. Calling the theorem requires a proof of each, so every assumption is visible where the theorem is used.

- **Bytes are natural numbers.** The models use `List Nat` for byte strings. Where a theorem needs real bytes, it says so with `(hBytes : ∀ byte ∈ bytes, byte < 256)`. In Rust the type `u8` guarantees this, so the hypothesis always holds for the real code.
- **Inversion lemmas.** Several lemmas start from "the function returned `some …`" and conclude what the input must have looked like. The proof follows the function's branches with `split at h`, throws away the branches that return `none`, and reads the facts off the branch that remains. `readArgument_accepted_facts`, `decodeHead_extended_facts`, `parked_reset_was_requested_in_window` and `unwrap_accepts_only_wrapped_ids` all work this way.

10.2 BYTES AND NUMBERS

File: `formal/Jotakey/Cbor/Heads.lean`, first half. CBOR stores multi-byte arguments big-endian. Before we can say anything about heads, we need four conversion functions and the facts relating them.

HEADS.LEAN: THE CONVERSION FUNCTIONS

```
def toLittleEndian : Nat → Nat → List Nat    -- the low `width` bytes of `value`
  | 0,         _     => []
  | width + 1, value => value % 256 :: toLittleEndian width (value / 256)

def fromLittleEndian : List Nat → Nat        -- little-endian bytes to a number
  | [] => 0
  | byte :: tail => byte + 256 * fromLittleEndian tail

def toBigEndian (width value : Nat) : List Nat := (toLittleEndian width value).reverse
def fromBigEndian (bytes : List Nat) : Nat := fromLittleEndian bytes.reverse
```

The big-endian functions are defined through the little-endian ones and `reverse`. That is a deliberate modelling choice. Little-endian functions peel one byte off the front of a list, which is exactly the shape of a list induction. Every big-endian fact then follows from its little-endian twin plus the library facts that `reverse` keeps length and membership and undoes itself. Eight theorems come in four pairs:

THEOREMS	STATEMENT
`toLittleEndian_length`, `toBigEndian_length`	The conversion always produces exactly `width` bytes.
`littleEndian_number_roundtrip`, `bigEndian_number_roundtrip`	If `value < 256 ^ width`, converting it to `width` bytes and back gives `value`: a number that fits survives the round trip.
`fromLittleEndian_fits_width`, `fromBigEndian_fits_width`	For a list of bytes, the number it holds is `< 256 ^ bytes.length`: `k` bytes always hold a number that fits in `k` bytes.
`littleEndian_bytes_roundtrip`, `bigEndian_bytes_roundtrip`	For a list of bytes, converting to a number and back to the same width gives the same list: bytes survive the round trip.

`toLittleEndian_length`. Induction on `width`, for every `value` (`generalizing value`, because the recursive call is at `value / 256`, not `value`). For `0` the list is empty. For `width + 1` the list is one byte followed by `toLittleEndian width (value / 256)`, whose length the induction hypothesis gives as `width`. **`toBigEndian_length`** is one `simp` call: `toBigEndian` reverses, reversing keeps length, done. Used by `readArgument_reads_encoded`.

`littleEndian_number_roundtrip`. Its hypothesis, that `value` fits in `width` bytes, is essential. For example, one byte of 1000 is `toLittleEndian 1 1000 = [232]`, which reads back as 232, not 1000: without the hypothesis the high bytes are cut off. The proof is induction on `width`:

HEADS.LEAN

```
theorem littleEndian_number_roundtrip (width value : Nat) (hFits : value < 256 ^ width) :
    fromLittleEndian (toLittleEndian width value) = value := by
  induction width generalizing value with
  | zero => simp [toLittleEndian, fromLittleEndian]; omega      -- value < 1, so value = 0
  | succ width ih =>
    have hQuotientFits : value / 256 < 256 ^ width := by       -- the rest fits in width bytes
      rw [Nat.pow_succ] at hFits
      exact (Nat.div_lt_iff_lt_mul (by decide)).mpr hFits
    simp only [toLittleEndian, fromLittleEndian, ih _ hQuotientFits]
    omega                                -- value % 256 + 256 * (value / 256) = value
```

In the `succ` case the induction hypothesis can only be used for numbers that fit in `width` bytes, so the proof first shows that `value / 256` does (`hQuotientFits`). Unfolding one step of each function, and replacing the inner round trip by `value / 256`, leaves `value % 256 + 256 * (value / 256) = value`, which is division with remainder, and `omega` knows it. **`bigEndian_number_roundtrip`** reduces to it because the two `reverse`s cancel. Used by `readArgument_reads_encoded`.

`fromLittleEndian_fits_width`. Induction on the list. The empty list gives `0 < 1`. For `byte :: tail`, the induction hypothesis says the tail's number `T` is below `256 ^ tail.length`, which for natural numbers is `T + 1 ≤ 256 ^ tail.length`. Multiplying by 256, and using `byte < 256`: `byte + 256 · T < 256 · (T + 1) ≤ 256 · 256 ^ tail.length`. The proof writes these steps as `have`s and lets `omega` combine them. One detail shows how `omega` works: it treats `256 ^ tail.length` as an unknown, opaque number. `256 * x` and `x * 256` are then different terms to it, so the proof first rewrites one into the other with `Nat.mul_comm`. **`fromBigEndian_fits_width`** applies it to the reversed list. Used, through `prefix_value_fits_width`, in the canonical-uniqueness proof: an argument read from `k` bytes fits back into `k` bytes.

`littleEndian_bytes_roundtrip`. Induction on the list again. For `byte :: tail` the number is `byte + 256 · T`. Because `byte < 256`, its remainder modulo 256 is `byte` and its quotient is `T` (two `omega` facts, `hRemainder` and `hQuotient`). So `toLittleEndian` emits `byte`, then converts `T` back into `tail.length` bytes, which is `tail` by the induction hypothesis. **`bigEndian_bytes_roundtrip`** is its big-endian twin. Used, through `reencoded_prefix_restores_bytes`, in the canonical-uniqueness proof.

10.3 CBOR HEADS

File: `formal/Jotakey/Cbor/Heads.lean`, second half. A **head** starts every CBOR item: an initial byte `majorType · 32 + additionalInfo` and, for additional info 24 to 27, an argument of 1, 2, 4 or 8 big-endian bytes (§5.1). The model has three functions:

HEADS.LEAN: ENCODER AND DECODER

```
def encodeHead (majorType argument : Nat) : List Nat :=      -- always the shortest form
  if argument < 24 then [majorType * 32 + argument]
  else if argument < 2 ^ 8 then (majorType * 32 + 24) :: toBigEndian 1 argument
  else if argument < 2 ^ 16 then (majorType * 32 + 25) :: toBigEndian 2 argument
  else if argument < 2 ^ 32 then (majorType * 32 + 26) :: toBigEndian 4 argument
  else (majorType * 32 + 27) :: toBigEndian 8 argument

def readArgument (width minimum : Nat) (bytes : List Nat) : Option (Nat × List Nat) :=
  if bytes.length < width then none                              -- truncated
  else if fromBigEndian (bytes.take width) < minimum then none   -- not minimal
  else some (fromBigEndian (bytes.take width), bytes.drop width)

def decodeHead : List Nat → Option (Nat × Nat × List Nat)
  | [] => none
  | initialByte :: rest =>
    let majorType := initialByte / 32
    let additionalInfo := initialByte % 32
    if majorType = 7 ∧ 24 ≤ additionalInfo ∧ additionalInfo ≤ 27 then none   -- floats
    else if additionalInfo < 24 then some (majorType, additionalInfo, rest)
    else if additionalInfo = 24 then
      (readArgument 1 24 rest).map fun (value, after) => (majorType, value, after)
    -- … the same for 25, 26, 27 with widths 2, 4, 8 and minimums 2^8, 2^16, 2^32
    else none                                          -- reserved, indefinite
```

The `minimum` argument of `readArgument` is the minimality rule: a value read from the 2-byte form must be at least 2^8, because anything smaller has a shorter encoding. That single comparison is what the canonical-uniqueness theorem depends on.

The round trip

`readArgument_reads_encoded`. Suppose the value is at least `minimum` and fits in `width` bytes. Then reading its encoding followed by `rest` returns the value and leaves exactly `rest`. The encoding has length `width` (`toBigEndian_length`), so `take width` returns it and `drop width` returns `rest` (two `have`s using library lemmas about `take` / `drop` of an append). Its value is `value` (`bigEndian_number_roundtrip`). Both rejection tests then fail: the input is long enough, and `value ≥ minimum`.

`decodeHead_inverts_encodeHead` (key theorem). For `majorType < 7` and `argument < 2^64`:

HEADS.LEAN

```
theorem decodeHead_inverts_encodeHead (majorType argument : Nat) (rest : List Nat)
    (hDataType : majorType < 7) (hFits64 : argument < argumentLimit) :
    decodeHead (encodeHead majorType argument ++ rest) = some (majorType, argument, rest)
```

The proof has five cases, chosen with `by_cases` to match the five branches of `encodeHead`: `argument < 24`, `< 2^8`, `< 2^16`, `< 2^32`, and the rest. Each case has the same three steps:

1. The initial byte `majorType · 32 + info` splits back into `majorType` (division by 32) and `info` (remainder), proved by `omega` because `info < 32` (`hMajor`, `hInfo`).
2. For the extended forms, `readArgument_reads_encoded` reads the argument back. Its hypothesis `minimum ≤ argument` holds because the case excludes the smaller values. For example, in the 2-byte case we know `¬ argument < 2^8` (the negation of `hOneByte`).
3. A final `simp; omega` clears the remaining tests in `decodeHead`. The float test `majorType = 7 ∧ …` is false because of `hDataType`.

The hypothesis `hDataType` excludes major type 7 because the decoder rejects info 24 to 27 there (floats). Jotakey only uses major-type-7 simple values below 24 (`false`, `true`, `null`), which the decoder does accept, but the theorem does not cover them. `hBytes` is not needed here: `encodeHead` always produces bytes, since `toBigEndian` takes every byte modulo 256.

Canonical uniqueness

The converse direction, from accepted bytes to the encoder, needs more scaffolding. Its lemmas each answer one question the main proof will ask:

- **`readArgument_accepted_facts`**: if `readArgument` returned `some (value, after)`, then the input had at least `width` bytes, `minimum ≤ value`, `value` is the number in the first `width` bytes, and `after` is what follows them. An inversion lemma. `split at h` follows the two `if`s. In the rejecting branches `h` claims `none = some (value, after)`, which `simp` recognises as impossible. In the accepting branch `obtain ⟨rfl, rfl⟩` substitutes `value` and `after` by the expressions they equal, and what is left is already known.
- **`decodeHead_extended_facts`**: the same for the four extended branches of `decodeHead`. If one of them accepted, giving `(majorType, argument, rest)`, then `majorType = initialByte / 32` and the four facts above hold. The proof looks at what `readArgument` returned (`cases hRead : …`); `none` cannot map to `some`.
- **`prefix_value_fits_width`**: for a byte list with at least `width` elements, the number in its first `width` bytes is `< 256 ^ width`. This is `fromBigEndian_fits_width` applied to that prefix.
- **`reencoded_prefix_restores_bytes`**: for the same list, re-encoding the first `width` bytes and appending the rest gives the list back (`bigEndian_bytes_roundtrip`, then `take ++ drop` is the whole list).
- **`encodeHead_one_byte_form`, `_two_byte_form`, `_four_byte_form`, `_eight_byte_form`**: given the range of `argument`, which branch `encodeHead` takes. For example, if `2^8 ≤ argument < 2^16`, then the encoding is `(majorType · 32 + 25)` followed by `toBigEndian 2 argument`. Each proof unfolds `encodeHead`, splits

every `if` (`repeat' split`), and closes each branch with `rfl` (the matching branch) or `omega` (the branch conditions contradict the range).

`decodeHead_accepts_only_canonical` (key theorem):

HEADS.LEAN

```
theorem decodeHead_accepts_only_canonical (bytes : List Nat) (majorType argument : Nat)
    (rest : List Nat) (hBytes : ∀ byte ∈ bytes, byte < 256)
    (h : decodeHead bytes = some (majorType, argument, rest)) :
    bytes = encodeHead majorType argument ++ rest
```

The proof follows `decodeHead`'s chain of `if`s with `split at h`, one branch at a time:

- **Rejected branches** (floats, info 28 to 31): `h` says `none = some …`. Impossible.
- **Immediate argument** (info < 24): `majorType` and `argument` are the quotient and remainder of the initial byte by 32, and `rest` is the tail. `encodeHead` takes its first branch because `argument < 24`, and quotient · 32 + remainder is the initial byte again.
- **Extended argument** (info 24 to 27, so `width` is 1, 2, 4 or 8): by `decodeHead_extended_facts`, the argument is the number in the next `width` bytes and is at least `minimum`; by `prefix_value_fits_width`, it is below `256 ^ width`. That is exactly the range where the matching `encodeHead_…_form` lemma applies, so the goal becomes: the input is the initial byte, then `toBigEndian width argument`, then the rest. By `reencoded_prefix_restores_bytes`, re-encoding the argument gives back the bytes it was read from. `congr 1` leaves only the initial bytes to compare, and `omega` checks them.

The proof does not say "the decoder is strict" anywhere. Strictness is **forced** by the statement. Take the non-minimal head `19 00 05`: 5 written in two bytes. A decoder without the minimality check would accept it as `(0, 5)`, but `encodeHead 0 5 = [05]`, so the theorem would be false and no proof would exist. In the info-25 case, the proof uses `hMinimal : 2⁸ ≤ argument` to apply `encodeHead_two_byte_form`, and that fact comes only from the minimality check. §9.8 shows the differential test catching exactly this bug in Rust.

The hypothesis `hBytes` is needed as well. Without it, `[0x18, 300]` (a "byte" of 300) decodes as `(0, 300)`, but `encodeHead 0 300` is the three bytes `[0x19, 1, 44]`. In Rust the type `u8` makes `hBytes` hold automatically.

`decodeHead_consumes_input` (key theorem) says an accepted head always leaves less input than it found: if `decodeHead` returns `rest`, then `rest.length < bytes.length`. The proof is the same case analysis, simpler: an immediate head leaves the tail after one byte, and an extended one leaves `tail.drop width`, no longer than the tail. The proof ends with `all_goals first | (simp at h; done) | (…)`: on every remaining branch, first try to show the branch is impossible, and if that does not close it, use `decodeHead_extended_facts` and arithmetic. Every head consumes input, so a loop over heads always moves forward. In Rust this is the reason an array claiming 2^{64} elements runs out of input after at most as many iterations as there are bytes. The model does not yet state that work bound for the whole validator; this lemma is the step it would be built from.

10.4 THE STRICT VALIDATOR

File: `formal/Jotakey/Cbor/Cbor.lean`. This file defines the whole validator (`skipItem`, `skipArray`, `skipMap`, `validate`, with helpers such as `readMapKey`, `canonicalKeyLess` and `isValidUtf8`) on top of the heads, and contains **no theorems**. Two guarantees come from it anyway.

Termination. The three `skip` functions call each other (an array contains items, an item may be an array), so they are defined together in a `mutual` block. Lean accepts them only after checking that every recursive call decreases a measure. Here the measure is a pair `(fuel, count)`, compared lexicographically. Entering a container lowers `fuel`; moving to the next element of an array or map keeps `fuel` and lowers the count; and an item (count 0) is always below the array that contains it (count `count + 1`). The definition compiling is therefore a proof that `validate` terminates on every input.

An executable specification. `validate` and `decodeHead` are what `jotakey-spec` runs in the differential test (§9.8). Agreement on 101,536 queries ties the Rust decoder to the same definitions the head theorems are

about. The nine `#guard` lines at the end of the file are build-time checks on hand-picked inputs: correct and misordered maps, duplicate keys, depth 4 and 5, a non-minimal head, an indefinite length, a float, and `true` followed by a trailing byte.

Which properties of the validator are **not** proved? Map-key order, rejection of duplicate keys, the depth limit, and UTF-8 validity are checked by the `#guard`s, the differential test, and the Rust tests and fuzzers. No theorem about them exists. They would be natural next theorems: for instance, "if `validate` accepts, the keys of every map are strictly increasing in canonical order".

10.5 CTAPHID REASSEMBLY

File: `formal/Jotakey/CtapHid.lean`. A `ReceiveState` is `idle`, `receiving` (owner channel, command, announced total, buffer so far, next expected sequence number) or `busy` (a complete message waiting for the authenticator). `receivePacket` handles one `ParsedPacket` (a packet already split into its fields), and `receivePackets` folds it over a list of packets.

Isolation

`receivePacket_ignores_other_channels`: if channel `owner` owns the transaction (`hOwner : state.owner = some owner`) and the packet is on another channel (`hOtherChannel`), then `receivePacket state packet = state`. The proof is a case analysis on the state. `idle` has no owner, so `hOwner` is false. In `receiving`, every branch of `receivePacket` first compares the packet's channel with the owner, and `hOtherChannel` says they differ, so `simp` picks the branch that returns the state unchanged. In `busy`, `receivePacket` ignores every packet, and `cases packet <;> rfl` checks both packet kinds.

`receivePackets_ignores_other_channels` (key theorem): if `owner` owns the transaction and every packet in the list is on another channel, then receiving them all leaves the state unchanged. Induction on the list: the first packet changes nothing (`receivePacket_ignores_other_channels`), so the rest runs from the same state. Any number of foreign packets, of any kind, in any order, are invisible to the owner. The model has no outputs or timers. The "channel busy" errors the Rust code sends back to the other channels are outside it.

Bounded memory

`WellFormed` is a predicate on receive states: in `receiving`, the buffer is shorter than the announced total and the total is at most 1200; in `busy`, the message is at most 1200 bytes.

- **`deliverIfComplete_is_well_formed`**: `deliverIfComplete`, which delivers a message once all its bytes are in, produces a well-formed state from a buffer no longer than its total. Either the buffer is still short (still receiving), or it is delivered and `buffer.length ≤ total ≤ 1200`.
- **`receivePacket_preserves_well_formed`**: one packet preserves `WellFormed`. Case analysis on state and packet. The two cases where data is stored need an argument. An init packet is refused if it announces more than 1200 bytes; otherwise it stores at most `announcedLength` bytes of its payload. A continuation packet stores at most `total - buffer.length` more bytes. In both, `deliverIfComplete_is_well_formed` applies.
- **`receivePackets_preserves_well_formed`** (key theorem): any packet sequence preserves `WellFormed`. Induction, with `receivePacket_preserves_well_formed` at each packet.

Note what makes the proof work: the device cuts every payload to what it still expects, whatever the packet claims. A model that appended the full 59 bytes of every continuation packet would make `receivePacket_preserves_well_formed` false at the last packet of a message.

Exact reassembly

The host side is modelled too. `fragmentMessage channel command message` builds the packets a host must send: an init packet with the first 57 bytes, then `continuationPackets` with 59 bytes each and sequence numbers 0, 1, 2 and so on.

- **`trimming_keeps_host_payload`**: `(bytes.take n).take (min bytes.length n) = bytes.take n`. The device cuts each payload to the bytes it still expects; this lemma says that cut removes nothing from a packet the host built.
- **`continuationPackets_of_empty`**: no bytes left means no continuation packets.

- **`continuations_complete_message`**: the core lemma. If the device is receiving `message` and holds a prefix `buffer`, with `buffer ++ rest = message` and `buffer` shorter than `message`, then the host's continuation packets for `rest` take it to `busy channel message`.

`continuations_complete_message` is proved by **calling itself**. That is how Lean expresses an induction that is not over a plain list or number, here the length of `rest`. `termination_by rest.length` names the measure, and `decreasing_by` proves it drops: the recursive call uses `rest.drop 59`, shorter because `rest` is not empty (`hRestNonEmpty`). The body handles one packet. The device appends `rest.take 59` (`trimming_keeps_host_payload` shows the cut is harmless). If the buffer is still short, the remaining packets are exactly those for `rest.drop 59` with the next sequence number, and the recursive call finishes. Otherwise `rest` fitted in this one packet, the buffer is `buffer ++ rest = message`, and no packets remain.

`fragmented_message_is_reassembled` (key theorem) says that any message of at most 1200 bytes, fragmented the way a host must fragment it, arrives intact:

CTAPHID.LEAN

```
theorem fragmented_message_is_reassembled (channel command : Nat) (message : List Nat)
    (hMaxSize : message.length ≤ maxMessageSize) :
    receivePackets .idle (fragmentMessage channel command message) = .busy channel message
```

The init packet is accepted and stores `message.take 57`. If that is the whole message, it is delivered at once. Otherwise `continuations_complete_message` applies, with `buffer = message.take 57` and `rest = message.drop 57`. This includes the empty message: an early version of the theorem required `0 < message.length`, and Lean showed that hypothesis was never used (§9.6).

10.6 THE AUTHENTICATOR STATE MACHINE

File: `formal/Jotakey/Authenticator.lean`. Parsing and cryptography are abstracted away. A request arrives as a `Command` already classified by the parser, and a signature is the output `signature relyingParty counter`. Relying parties and credentials are plain numbers: the model only needs to tell them apart. `processRequest` handles a request and may park a command that needs a touch; `completePresence` handles the outcome of the wait for a touch; `handleEvent` dispatches one event to either, and `runTrace` runs any list of events. The `State` holds the `pending` command, the persisted `signCounter`, the `resetEpoch` and the `faulted` flag.

Most single-event proofs have the same shape: split into every combination of state and input (`cases … <;> …`), let `simp` compute what `processRequest` or `completePresence` returns, and see that the claim holds in each case. There are dozens of cases, and Lean checks all of them. If a command is ever added to `Command`, `cases` includes it automatically, and the proof fails if the new command breaks the property.

User presence (invariant 1)

- **`processRequest_never_signs_with_presence`**: whatever the state, command and uptime, `processRequest` never outputs a signature carrying the UP flag. The proof tries faulted or not × every command, plus the uptime `if` for reset; every output is an error, `info`, `needsPresence`, or (for a silent assertion) a signature with the flag `false`.
- **`request_signs_only_silently`**: the only request that signs at all is a silent assertion on a healthy device, for its own relying party, and its signature has the UP flag clear. This is the model of the client preflight (§5.2).
- **`completePresence_signs_with_presence`**: every signature a completed touch produces carries the UP flag.
- **`completePresence_clears_pending`**: after `completePresence`, nothing is parked, whatever the outcome. Every branch sets `pending := .idle`.
- **`signature_requires_confirmed_touch`**: if an event outputs a signature carrying the UP flag for `relyingParty`, then the event was `presence .confirmed`, the device was not faulted, and the parked command was a makeCredential or getAssertion **for that same relying party**. A request cannot sign

with the flag (`processRequest_never_signs_with_presence`). For a presence event, the proof tries every parked command × every result (18 cases, `simp_all` computes each). Only the two confirmed signing cases produce a signature, and there the relying party is the parked one.

- **`one_touch_signs_at_most_once`** (key theorem): `completePresence` applied twice never signs the second time, with or without the flag. Suppose it did; the signature would carry the flag (`completePresence_signs_with_presence`). `signature_requires_confirmed_touch` says a signing command was parked before the second completion, but `completePresence_clears_pending` says nothing was. `rcases` takes the two possibilities apart (makeCredential, or getAssertion with some credential), and in each `cases` sees that `.idle` cannot equal a different constructor.

The signature counter (invariant 2)

- **`handleEvent_never_lowers_counter`** : one event never lowers `signCounter` . Every branch keeps it or adds 1.
- **`signature_carries_next_counter`** : if an event signs with `counter` , then `counter = state.signCounter + 1` and the new state's `signCounter` is `counter` . In the Rust code this is the write-ahead rule: `N + 1` is persisted before the signature that carries it is released.
- **`runTrace_never_lowers_counter`** : over a whole trace, the final counter is at least the start. Induction, chaining single events with `Nat.le_trans` .
- **`trace_signature_counters_bounded`** : every signature `counter` in a trace satisfies `start < counter ≤ final` .
- **`trace_signature_counters_increase`** (key theorem): the signature counters of any trace are `Pairwise (· < ·)` , so each is smaller than every counter that comes after it.

AUTHENTICATOR.LEAN

```
theorem trace_signature_counters_increase (state : State) (events : List Event) :
    (signatureCounters (runTrace state events).2).Pairwise (· < ·)
```

`trace_signature_counters_bounded` exists only to make this proof go through, and it shows a technique worth knowing: **strengthening the induction hypothesis**. Try to prove `trace_signature_counters_increase` directly by induction on the events. The tail of the trace is pairwise increasing by the induction hypothesis. But if the first event signs with counter `first` , we must also show that `first` is below every **later** counter, and "pairwise increasing" says nothing about where the later counters lie. So we first prove the stronger `trace_signature_counters_bounded` , "every counter lies strictly above the start and at most at the end", and that is enough. The first signature's `first` is the counter right after the first event (`signature_carries_next_counter`). Every later signature lies strictly above that counter, because the bounded lemma applies to the rest of the trace, which starts there. Proving something stronger to make an induction work is very common.

The trace is **any** list of events: requests of every kind, touches, timeouts, cancellations and resets, in any order. A reset changes the epoch but not the counter, so counters keep increasing across resets too, as relying parties require.

Reset (invariant 6) and faults (invariant 7)

- **`parked_reset_was_requested_in_window`** : if processing a request leaves a reset parked, the request was `reset` with `uptimeMs ≤ 10000` . Inversion over every command and the uptime `if` .
- **`epoch_change_requires_confirmed_touch`** : if an event changes the reset epoch, the event was a confirmed touch and a reset was parked. Requests never change the epoch. The proof shows that case by case, after `exfalso; apply hChanged` turns "find the contradiction" into "show the epochs are equal". For presence events, only reset/confirmed changes it.
- **`epoch_change_requires_reset_in_window`** (key theorem): start from a state with no reset parked (the hypothesis `hNoParkedReset` ; power-up has none). If the epoch ever changes, the trace contains a reset request whose uptime was at most 10,000 ms. Induction on the events. If the first event changed the epoch, `epoch_change_requires_confirmed_touch` says a reset was parked at the start, which contradicts

`hNoParkedReset`. Otherwise the change happens later, and to use the induction hypothesis we need "no reset parked" after the first event. A touch leaves nothing parked (`completePresence_clears_pending`). A request that parked a reset **is** the witness we are looking for (`parked_reset_was_requested_in_window` supplies the bound). Any other request leaves no reset parked, and the induction hypothesis finds the witness further on.

- **`faulted_stays_faulted_and_never_signs`** (key theorem): if `faulted` is set, then after any event it is still set and the output is not a signature. Both `processRequest` and `completePresence` take their first branch; `simp` sees it directly. Applying it at each event in turn shows that a faulted device never signs again in any trace. That induction is not written out as a separate theorem.

10.7 CREDENTIAL IDS

File: `formal/Jotakey/Credential.lean`. A credential ID is `[1] ++ nonce ++ mac key relyingParty nonce`. The MAC is a **parameter**, declared once with `variable (mac : …)`: every theorem holds for any function of that type, HMAC-SHA256 included, and states as a hypothesis whatever it needs from it.

- **`unwrap_recovers_nonce`** (key theorem): assume the MAC always returns 16 bytes (the hypothesis `hTagLength`) and the nonce has 32. Then unwrapping a freshly wrapped ID at the same relying party returns the nonce. Direct computation: the length is 1 + 32 + 16, the version byte matches, splitting after 32 bytes gives back the nonce and the tag, and the recomputed tag equals the stored one.
- **`unwrap_accepts_only_wrapped_ids`**: if `unwrap` accepts `credentialId` and returns `nonce`, then `credentialId = wrap mac key relyingParty nonce` and the nonce has 32 bytes. No assumption about the MAC. An inversion proof: the accepting branch says the version is right, the length is 49, and the recomputed MAC of `rest.take 32` equals `rest.drop 32`. Putting the pieces back together (`List.take_append_drop`) gives exactly `wrap`'s output. `unwrap` accepts nothing else: not a different version byte, not a longer ID with trailing bytes, not a near miss.
- **`credential_bound_to_relying_party`** (key theorem):

CREDENTIAL.LEAN

```
theorem credential_bound_to_relying_party
    (hNoCrossSiteCollision : ∀ key party party' nonce,
      party ≠ party' → mac key party nonce ≠ mac key party' nonce)
    (key relyingParty otherParty nonce : List Nat) (hNonceLength : nonce.length = nonceLength)
    (hDifferentParty : relyingParty ≠ otherParty) :
    unwrap mac key otherParty (wrap mac key relyingParty nonce) = none
```

The proof is by contradiction. Suppose `unwrap` at `otherParty` returned some `recovered` nonce. By `unwrap_accepts_only_wrapped_ids` the ID also equals `wrap` at `otherParty` for `recovered`: the nonce followed by its tag at `relyingParty` equals `recovered` followed by its tag at `otherParty`. Both nonces are 32 bytes, so `List.append_inj` splits the equation into `nonce = recovered` and equal tags for the two relying parties, which `hNoCrossSiteCollision` rules out because they differ.

> **THE ASSUMPTION IS AN IDEALISATION**
> Stated for **all** inputs, `hNoCrossSiteCollision` is false for any real 16-byte MAC. There are more relying parties than tags, so some pair must collide. What cryptography provides is that nobody can **find** such a collision, except with probability about 2^{-128} per attempt. Read the theorem as: a MAC collision is the only way a credential ID can be accepted at a site it was not minted for. The ID format, the parsing and the comparison add no other way. Proving the probabilistic statement itself would need a framework for cryptographic proofs, such as EasyCrypt or SSProve, and is outside the scope of this project.

10.8 WHAT THE PROOFS LEAVE OUT

Each model makes choices about what to leave out (§9.4), and those choices are as important as the theorems. In one place:

MODEL	NOT COVERED BY A THEOREM
CBOR	The validator's map-key order, duplicate-key rejection, depth limit and UTF-8 checks. They are checked by `#guard` s, the differential test and the Rust tests and fuzzers, not proved. Major type 7 is outside the head round trip.
CTAPHID	`INIT` , `CANCEL` , timeouts and the response path. A combined theorem — owner packets interleaved with foreign ones give the same result as the owner packets alone — would follow from the existing lemmas but is not stated.
Authenticator	The parser's classification of requests into `Command` , the clock that provides `uptimeMs` , flash write failures, and keepalives. Fault absorption is proved per event, not over traces.
Credentials	The cryptography: the MAC is a parameter, and `hNoCrossSiteCollision` idealises collision resistance. Key derivation (HKDF, P-256 scalars) is not modelled.
Link to Rust	Only the CBOR model is tied to the code by differential testing. The others are tied by careful transcription, review, and the tests and fuzz oracles of chapter 8 that check the same properties on the real code (§9.7). The planned Aeneas experiment would replace transcription with translation.

None of these gaps is hidden: each is either a hypothesis in a theorem's signature or a part of the system that no theorem mentions. That is the standard to keep as the models grow. When a new feature is added, its invariant goes into the model first, the proof goes into `AxiomAudit.lean` , and anything the proof assumes stays visible in the signature.

10.9 RUST AND LEAN NAMES

Each side follows its own language's conventions. Rust writes functions in `snake_case` and constants in `SCREAMING_SNAKE_CASE` ; Lean writes definitions in `lowerCamelCase` and names theorems in `snake_case` after what they state. Constants that quote the CTAP specification (`BROADCAST_CID` , `cmd::CBOR`) keep the spec's spelling in Rust. Everything else uses descriptive words on both sides, so most names differ only in casing. The tables below list every modelled item (Rust paths are under `crates/jotakey-core/src` , Lean paths under `formal/Jotakey`); `formal/NAMES.md` holds the same table next to the code, and `AGENTS.md` requires both to change together with any rename.

RUST	LEAN	NOTES
CBOR		
`Decoder::head` , `Encoder::head`	`decodeHead` , `encodeHead`	Rust advances a cursor; Lean returns the rest of the input.
`Decoder::read_be` + minimality check	`readArgument`	
`Decoder::skip(depth)`	`skipItem` , `skipArray` , `skipMap`	One function with loops; three mutually recursive ones.
`skip(0)` + `position()`	`validate`	Answers `validate <hex>` in `jotakey-spec` .
`read_slice` , `next_key` , `canonical_lt`	`takeExactly` , `readMapKey` , `canonicalKeyLess`	
`core::str::from_utf8` , `MAX_DEPTH`	`isValidUtf8` , `maxNestingDepth`	
CTAPHID		
`ReceiveState` (`Idle` , `Receiving` , `Busy`)	`ReceiveState` (`idle` , `receiving` , `busy`)	Same names.
`Receiving { channel, command, total, received, next_sequence, last_packet_ms }`	`receiving channel command total buffer nextSequence`	Rust counts the bytes received; Lean holds them. Lean has no clock.
`Busy { channel }`	`busy channel message`	Rust keeps the message in `receive_buffer` .

RUST	LEAN	NOTES
`CtapHid::handle_packet`, `maybe_dispatch`	`receivePacket`, `deliverIfComplete`	Lean leaves out INIT, CANCEL and timeouts.
`Packet` (raw `[u8; 64]`)	`ParsedPacket` (`init`, `continuation`)	Different on purpose: the model starts after parsing.
`MAX_MSG_SIZE`, `INIT_DATA`, `CONT_DATA`	`maxMessageSize`, `initPayloadSize`, `continuationPayloadSize`	
—	`receivePackets`, `WellFormed`, `fragmentMessage`, `continuationPackets`	Model only: packet sequences, the invariant, the host side.
Authenticator		
`Authenticator::process`, `complete`	`processRequest`, `completePresence`	Rust parses CBOR; Lean receives a classified `Command`.
`PresenceResult`	`PresenceResult`	Same name.
`Pending`	`PendingCommand`	Same variants.
`PendingOrSilent::Silent`	`Command.silentAssertion`	getAssertion with `up: false`: signed in `process`, never parked.
`sign(…, user_present, …)`	`Output.signature … userPresent`	The UP flag; `true` only from `complete` under `Confirmed`.
`Outcome` + response bytes	`Output`	
`Platform::increment_counter`, `rotate_epoch`, `uptime_ms`	`State.signCounter`, `State.resetEpoch`, `uptimeMs`	Rust keeps them behind the platform trait.
`RESET_WINDOW_MS`	`resetWindowMs`	
—	`Command`, `Event`, `handleEvent`, `runTrace`, `signatureCounters`	Model only.
Credential IDs		
`wrap`, `unwrap` (`tagger`, `rp_id_hash`, `id`)	`wrap`, `unwrap` (`mac key`, `relyingParty`, `credentialId`)	Lean takes the MAC as a parameter `mac`; Rust's `Tagger` plays `mac key`. Rust's `unwrap` can also fail when the hardware does, which the model leaves out.
`VERSION`, `NONCE_LEN`, `TAG_LEN`	`formatVersion`, `nonceLength`, `tagLength`	
return type `[u8; TAG_LEN]`	hypothesis `MacOutputsTagLength mac`	The Rust type guarantees what Lean must assume.

The properties line up the same way: the Rust tests and fuzz oracles check on the real code what the Lean theorems prove about the model.

RUST TEST OR FUZZ ORACLE	LEAN THEOREM
`head_roundtrip`	`decodeHead_inverts_encodeHead`
`accepted_heads_are_canonical`, `cbor_decode` oracle	`decodeHead_accepts_only_canonical`
`other_channel_gets_busy…`, `ctaphid_stack`	`receivePackets_ignores_other_channels`
`ctaphid_stack` buffer assertions	`receivePackets_preserves_well_formed`
`no_signature_without_presence`, `ctap_dispatch` oracle	`signature_requires_confirmed_touch`, `one_touch_signs_at_most_once`
`monotonic_across_power_cuts`	`trace_signature_counters_increase`
`reset_window_and_effect`	`epoch_change_requires_confirmed_touch`, `epoch_change_requires_reset_in_window`
`fails_closed`	`faulted_stays_faulted_and_never_signs`
`roundtrip`, `bound_to_rp` (`credential.rs`)	`unwrap_recovers_nonce`, `credential_bound_to_relying_party`

11 ROADMAP

The MVP is deliberately small: every line of it is tested, fuzzed, mutation-tested, and, for the security-critical state machines, proved. The next steps, in order, each follow the same discipline: model, prove, implement, test, fuzz, mutate.

11.1 FULL CTAP 2.1 PASSKEYS

Discoverable credentials ("username-less passkeys") need the authenticator to **store** something per credential, and need a PIN, because a discoverable credential with only presence would let anyone holding the key list your accounts. The steps:

1. **Resident-key store.** A fixed-capacity table in the `jotakey` partition (for example 64 entries). Each entry holds `rpIdHash`, the user handle (≤ 64 bytes), user name and display name (truncated), a creation counter, and the credential nonce. The private key is still derived, not stored. Entries live in a record log like the counter's, so writes stay power-safe. Lean: model the store as a finite map, and prove capacity bounds and that deletion really deletes.
2. **makeCredential with `rk: true`.** Store the entry **before** returning the response. Replace an existing entry for the same (`rpIdHash`, user handle). Return `CTAP2_ERR_KEY_STORE_FULL` when full. Advertise `options.rk = true`.
3. **getAssertion without allow list.** Find all entries for the RP, newest first. Return the first with `numberOfCredentials`, then serve `authenticatorGetNextAssertion` (0x08) from a short-lived iterator that a new command or a 30 s timeout invalidates. Include `user` in the response (only `id` unless UV was performed).
4. **clientPIN, protocol 2.** ECDH (P-256) key agreement, HKDF-derived shared secrets, AES-256-CBC for PIN transport, HMAC-SHA256 `pinUvAuthParam`. PIN hash stored as `LEFT(SHA-256(PIN), 16)` under the device key. **Retry counters persisted before checking**: 8 attempts in total, 3 per power cycle, then a forced reset. These lockout rules are a natural Lean theorem: no trace allows more than 8 wrong PIN guesses. Add `uv`, `clientPin` and `pinUvAuthToken` to getInfo.
5. **credentialManagement (0x0A).** Enumerate RPs and credentials, delete credentials, update user info. Requires a `pinUvAuthToken` with the `cm` permission.
6. **Extensions.** `credProtect` (levels 1–3: hide credentials from assertion without UV), `hmac-secret` (symmetric secrets for disk encryption and password managers, derived per credential with a salt from the platform), and later `largeBlob`.
7. **U2F / CTAP1 compatibility.** `U2F_REGISTER` and `U2F_AUTHENTICATE` over `CTAPHID_MSG`, for older relying parties. Clear the `NMSG` capability bit.
8. **Versions and conformance.** Advertise `FIDO_2_1` and pass the FIDO Alliance conformance tools for the implemented feature set.

11.2 HARDWARE

The jotakey key (§7.5) delivered the two largest hardware steps of the original plan: a secure element holding the root, generated on the device, and native USB without a bridge. What follows:

- **Firmware for the jotakey key.** The core already derives per credential (§6.3), on both platforms. Next come the STM32 adapters: USB HID, the touch sensor, the LED, the OPTIGA driver with its shielded connection, and the provisioning build.
- **Signed firmware updates (exploration).** Production keys are locked and never updated (§7.5). An alternative worth studying is a small, immutable bootloader that installs only firmware signed with an offline key. It would let fixes reach deployed keys, at the price of a bootloader that is itself security-critical code to design, test and prove.
- **OPTIGA Trust M V3**, once it is in stock: the same package and pinout, with HMAC, HKDF and AES. `F` would become HMAC-SHA256 on both platforms.
- **Signing inside the secure element**, which would stop a thief who breaks the microcontroller from using the key at all. It needs a secure element that can sign with derived keys, or a fixed-size store of per-credential keys.

- **Persistent BLE bonds** on the C6, in the encrypted partition, so phones and laptops do not have to re-pair after a power cycle.

11.3 ASSURANCE

- **Aeneas experiment** (§9.7): translate `ctaphid.rs` and `cbor.rs` to Lean with Charon/Aeneas, and prove that the generated functions refine the hand-written models. The core is already written in the supported Rust subset.
- **Kani harnesses:** prove absence of panics in the decoders and state machines for all inputs up to a bound, directly on the Rust code.
- **Lean models for the record log** (power-loss safety as a theorem over interleavings of writes and cuts) and for the BLE framer.
- **Reproducible firmware builds** and published hashes, so a user can check that the binary they flash is the one that was reviewed.
- **Side-channel evaluation** of the signing path with a ChipWhisperer-class setup, to put numbers on chapter 7.

01 INVARIANT TRACEABILITY

Every invariant of `AGENTS.md` §2, with where it is enforced, tested and proved.

#	ENFORCED IN	TESTED / FUZZED	PROVED (FORMAL/JOTAKEY/…)
1	`ctap/mod.rs` `complete` (only UP=1 signing path); silent `up:false` assertions signed with UP clear; `presence.rs` fresh debounced press	`no_signature_without_presence`, `processing_parks_and_complete_consumes`, `silent_assertion_signs_at_once_without_the_up_flag`; `ctap_dispatch` oracle	`Authenticator.signature_requires_confirmed_touch`, `request_signs_only_silently`, `one_touch_signs_at_most_once`
2	`increment_counter` before signing; `storage.rs` append + read-back (C6); OPTIGA up-counter (key)	`register_then_authenticate…`, `monotonic_across_power_cuts`, on-device torn write	`Authenticator.trace_signature_counters_increase`
3	`credential.rs` tag over rpIdHash, `ct_eq`	`bound_to_rp`, `any_bit_flip_is_rejected`, `credential_is_bound_to_rp`; `credential_unwrap`	`Credential.unwrap_accepts_only_wrapped_ids`, `credential_bound_to_relying_party`
4	`cbor.rs` strict decoder; `MAX_MSG_SIZE` everywhere; no-panic lints	`decoder_never_panics`, size boundaries; all fuzz targets; differential test	`Cbor.decodeHead_accepts_only_canonical`, `decodeHead_consumes_input`; `CtapHid.receivePackets_preserves_well_formed`
5	`ctaphid.rs` single transaction, owner checks	`other_channel_gets_busy…`, `hid_init_resync_only_touches…`; `ctaphid_stack`	`CtapHid.receivePackets_ignores_other_channels`, `fragmented_message_is_reassembled`
6	`ctap/reset.rs` window + touch; epoch rotation	`reset_window_and_effect`, `reset_window_boundary_is_inclusive`	`Authenticator.epoch_change_requires_confirmed_touch`, `epoch_change_requires_reset_in_window`
7	`main.rs` boot sequence; `Authenticator::fault`; `RngHealth` on every byte	`fails_closed`, `every_injected_fault_is_detected`, on-device KATs and TRNG	`Authenticator.faulted_stays_faulted_and_never_signs`
8	`Zeroizing` secrets; HMAC peripheral (C6) or OPTIGA (key); no `Debug` of secrets	Review; `cargo deny`	—
9	`subtle::ConstantTimeEq`; RustCrypto `p256`	Review	—

02 REPOSITORY MAP

PATH	CONTENTS
`AGENTS.md`	Normative requirements and security rules for anyone (human or AI) changing the code.
`crates/jotakey-core`	Protocol core; `tests/` (CTAP, transports, differential).
`crates/jotakey-fw`	Firmware; `partitions.csv`; `deny.toml` with documented exceptions.
`crates/jotakey-testfw`	On-device tests (`cargo test` with the board attached).
`crates/jotakey-bridge`	Linux serial ⇄ uhid bridge.
`crates/jotakey-host-tests`	Independent client, simulator, RP checks, hardware tests.
`fuzz/`	Six cargo-fuzz targets.
`formal/`	Lean 4: `Cbor`, `CtapHid`, `Authenticator`, `Credential`; `Main.lean` (executable spec); `AxiomAudit.lean`.
`scripts/`	`provision.sh` (irreversible, human-only), udev rules, `fido2_check.py`.
`docs/`	This report (`typst compile docs/jotakey.typ`).

03 GETTING STARTED

FROM CLONE TO A WORKING KEY (DEVELOPMENT BUILD)

```
rustup target add riscv32imac-unknown-none-elf
cargo install espflash probe-rs-tools cargo-fuzz cargo-mutants cargo-deny just
curl -sSf https://raw.githubusercontent.com/leanprover/elan/master/elan-init.sh | sh

just test && just lean && just diff          # host tests, proofs, Rust-vs-Lean
cd crates/jotakey-fw
cargo run --release --features dev-key       # INSECURE dev key; logs on the UART port
cd ../.. && just bridge /dev/ttyACM0         # needs /dev/uhid access (scripts/99-jotakey.rules)
python3 scripts/fido2_check.py               # or register at https://webauthn.io
```

04 GLOSSARY

TERM	MEANING
AAGUID	Authenticator Attestation GUID: 16 bytes identifying the authenticator model.
Assertion	The signed response of an authentication ceremony.
authData	authenticatorData: rpIdHash, flags, counter, optional credential data.
CBOR	Concise Binary Object Representation (RFC 8949).
COSE_Key	CBOR encoding of a public key (RFC 9052/9053).
CTAP	Client-to-Authenticator Protocol (FIDO Alliance).
CTAPHID	CTAP over USB HID: 64-byte reports, channels.
Discoverable credential	Resident key: stored on the authenticator with the user handle; a “passkey”.
KAT	Known-answer test: compute on fixed input, compare with a published result.
Relying party (RP)	The website or service that registers and verifies credentials.
RP ID	The domain a credential is scoped to.
UP / UV	User presence (a touch) / user verification (PIN or biometric).

TERM	MEANING
WebAuthn	W3C Web Authentication API, the browser side of FIDO2.

05 REFERENCES

- FIDO Alliance. **Client to Authenticator Protocol (CTAP)**, versions 2.0 (2019), 2.1 (2021), 2.2.
- W3C. **Web Authentication: An API for accessing Public Key Credentials**, Levels 2 and 3.
- C. Bormann, P. Hoffman. **RFC 8949: Concise Binary Object Representation**, 2020.
- T. Pornin. **RFC 6979: Deterministic Usage of DSA and ECDSA**, 2013.
- H. Krawczyk, P. Eronen. **RFC 5869: HKDF**, 2010.
- NIST. **SP 800-90B: Recommendation for the Entropy Sources Used for Random Bit Generation**, 2018.
- Espressif. **ESP32-C6 Technical Reference Manual**; **Security Advisory AR2023-007** (2024).
- W. M. McKeeman. **Differential Testing for Software**, Digital Technical Journal 10(1), 1998.
- X. Yang, Y. Chen, E. Eide, J. Regehr. **Finding and Understanding Bugs in C Compilers (Csmith)**, PLDI 2011.
- G. Klein et al. **seL4: Formal Verification of an OS Kernel**, SOSP 2009.
- X. Leroy. **Formal verification of a realistic compiler**, CACM 2009.
- J.-K. Zinzindohoué et al. **HACL*: A Verified Modern Cryptographic Library**, CCS 2017.
- A. Erbsen et al. **Simple High-Level Code for Cryptographic Arithmetic (Fiat-Crypto)**, S&P 2019.
- S. Ho, J. Protzenko. **Aeneas: Rust Verification by Functional Translation**, ICFP 2022.
- L. de Moura, S. Ullrich. **The Lean 4 Theorem Prover and Programming Language**, CADE 2021.
- E. W. Dijkstra. **Notes on Structured Programming**, 1970.